

**Mini Project - II Report
(MCA-S566)**

on

Band Blogging Website

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

**Master of Computer Applications (MCA)
Uttaranchal School of Computing Sciences**



**UTTARANCHAL
UNIVERSITY
DEHRADUN**

**NAAC
GRADE A+**

BATCH 2025-2027

Submitted to:

Mr. Ravindra Kumar

Assistant Professor

Uttaranchal School of Computing Sciences

Submitted by

Joshua Vanlalhnehpuia

Enrollment No: UU2520000116

MCA -II (C)

ACKNOWLEDGEMENT

The most awaited moment of any endeavor is its successful completion, but nothing can be done successfully if done alone. Success is the outcome of continuous technical mentoring, guidance, and support from various individuals, and I express my sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, I would like to thank **Prof. (Dr.) Sonal Sharma, Director, USCS**, for providing a healthy and encouraging environment for study.

I am extremely thankful to **Dr. Sameer Dev Sharma, Head of Department, USCS**, and my project mentor **Mr. Ravindra Kumar**, for providing me with this opportunity and for their valuable technical mentorship, continuous guidance, and expert insights throughout the project development process. Their support at every stage significantly enhanced my understanding and helped me successfully complete this work.

I also express my sincere gratitude to my colleagues for their valuable discussions and support, which helped me in giving a final shape to this project

Dated: 13th May, 2026

Joshua Vanlalhnehpuia

Enrollment No: UU2520000116

MCA – II (C)

STUDENT DECLARATION

I hereby declare that the project titled “**Band Blogging Website**” is an original work carried out by me under the guidance of **Mr. Ravindra Kumar** in partial fulfillment of the requirements for the award of the degree of **Master of Computer Application** from **Uttaranchal University**.

I further declare that this project has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

All sources of information and references used in this project have been duly acknowledged.

Dated: 13th May, 2026

Joshua Vanlalhnehpuia

Enrollment No: UU2520000116

MCA – II (C)

Uttaranchal School of Computing Sciences

CERTIFICATE

This is to certify that the project work entitled “**Band Blogging Website**” is a bonafide work carried out by **Joshua Vanlalhnehpuia (UU252000116)** submitted in partial fulfilment for the award of the degree in **MASTER OF COMPUTER APPLICATIONS** of **Uttaranchal University, Dehradun** during the academic year 2025-26.

It is verified that the project files and project report (PDF) are uploaded on GitHub <https://github.com/joji-equ/band-website>

Mr. Ravindra Kumar

Assistant Professor

USCS

TABLE OF CONTENT

ACKNOWLEDGEMENT.....	i
DECLARATION.....	ii
CERTIFICATE OF ORIGINALITY.....	iii
1. INTRODUCTION.....	1
2. OBJECTIVE.....	2
3. SYSTEM ANALYSIS.....	3
3.1 Identification of Need.....	3 - 4
3.2 Preliminary Investigation.....	4
3.3 Feasibility Study.....	4 - 6
3.4 Project Planning.....	7
3.5 Project Scheduling.....	7 - 10
3.6 Software Requirement Specifications (SRS).....	11
3.7 Software Engineering Paradigm.....	11 - 12
3.8 Data Models.....	12 - 17
4. SYSTEM DESIGN.....	18
4.1 Modularization.....	18 - 20
4.2 Data integrity and constraints.....	21 - 22
4.3 Database design.....	23 - 24
4.4 User Interface Design.....	25 - 31
5. TESTING.....	32
5.1 Testing techniques and Testing strategies used.....	32 - 33
5.2 Test reports for Unit Cases and System Test Cases.....	33 - 35
6. SYSTEM SECURITY MEASURES.....	36
7. COST ESTIMATION	37
8. REPORTS.....	38
9. FUTURE SCOPE.....	39
10. APPENDICES	
10.1 Coding.....	40 - 86
10.2 Bibliography.....	87

1. INTRODUCTION

A Band Blogging Website is a digital platform designed to connect music bands with their audience through engaging and dynamic content. In today's digital age, bands need more than just performances to grow their fan base—they require an online presence where they can share updates, stories, music releases, and interact with their followers. This project aims to provide a centralized space where bands can express their identity, promote their work, and build a strong community around their music.

The website allows bands to create and manage blogs that showcase their journey, upcoming events, albums, and behind-the-scenes experiences. It serves as both a promotional tool and a communication medium, enabling fans to stay updated and feel more connected to their favorite bands. Features such as user-friendly interfaces, comment sections, and multimedia integration enhance the overall user experience, making the platform interactive and engaging.

This project also focuses on simplicity, accessibility, and scalability. By implementing modern web technologies, the Band Blogging Website ensures smooth performance and easy navigation for both band members and users. Ultimately, the goal of this project is to bridge the gap between artists and audiences, providing a platform that not only promotes music but also fosters creativity, collaboration, and community building.

2. OBJECTIVE

The primary objective of the Band Blogging Website is to provide a dedicated online platform where music bands can share their content, including blogs, updates, and media, with their audience. It aims to help bands establish a strong digital presence and communicate effectively with their fans.

Another key objective is to create an interactive environment where users can engage with band content through comments, likes, and shares. This enhances the connection between artists and their audience, making the experience more personal and engaging.

The project also focuses on developing a user-friendly and responsive system that allows easy navigation and content management. Bands should be able to post and update information effortlessly, while users can access content anytime and from any device.

Lastly, the objective is to ensure scalability and performance by using modern technologies, so the platform can handle multiple users and grow over time as more bands and audiences join the system.

3. SYSTEM ANALYSIS

System analysis is the process of understanding the requirements and defining how the proposed system will function. In the case of the Band Blogging Website, the analysis focuses on identifying the needs of both bands (content creators) and users (audience). Bands require a platform to easily create, edit, and manage blog posts, upload media such as images and videos, and share updates about their music and events. On the other hand, users need a simple and engaging interface to browse content, read blogs, and interact through comments or feedback.

The existing system mainly relies on general social media platforms, which do not provide a dedicated space specifically for bands to showcase their journey and creative work in an organized manner. These platforms often mix different types of content, making it difficult for fans to find focused updates about a particular band. Therefore, there is a need for a specialized system that centralizes band-related content and offers better control and customization for artists.

The proposed system addresses these limitations by offering a structured blogging platform tailored for bands. It includes features such as user authentication, blog management, multimedia support, and user interaction modules. The system is designed to be user-friendly, responsive, and scalable, ensuring smooth performance and accessibility across devices. Overall, the analysis concludes that the development of this system will improve communication between bands and their audience while providing a dedicated and efficient platform for content sharing.

3.1 IDENTIFICATION OF NEED

In today's digital world, music bands rely heavily on online platforms to promote their work and connect with their audience. However, most existing platforms such as social media sites are not specifically designed for bands, which makes it difficult to organize and present their content in a focused and professional manner. Posts often get lost among unrelated content, limiting visibility and meaningful engagement with fans.

There is a clear need for a dedicated platform where bands can share their journey, publish blogs, promote events, and showcase their music in a structured way. Such a system would allow

bands to maintain their unique identity while providing fans with a centralized place to access all updates and content related to their favorite bands.

Additionally, fans are increasingly looking for more interactive and personalized experiences. A Band Blogging Website fulfills this need by enabling direct interaction through comments, feedback, and regular updates. Therefore, the development of this platform is essential to bridge the gap between bands and their audience, while improving content accessibility, organization, and engagement.

3.2 PRELIMINARY INVESTIATION

The preliminary investigation of the Band Blogging Website project was conducted to assess its feasibility, scope, and practical value. This involved analyzing current platforms used by bands, identifying their limitations, and understanding user expectations through basic research and observations. It was found that while general social media platforms allow content sharing, they lack structure, customization, and a focused environment for bands. The investigation also considered technical feasibility, confirming that the system can be developed using modern web technologies with manageable resources and cost. Additionally, operational feasibility was evaluated, showing that both bands and users can easily adapt to such a platform due to its simple and user-friendly design. Overall, the investigation indicates that the project is feasible, useful, and capable of fulfilling the identified needs effectively.

3.3 FEASIBILITY STUDY

The purpose of a feasibility study is to determine the viability of a proposed project. Feasibility study is carried out for a number of purposes, including establishing if software is suitable for development, installation, and business contribution.

Types are:

Technical Feasibility:

The Band Blogging Website is technically feasible as it can be developed using widely

available and well-supported technologies such as HTML, CSS, JavaScript, and backend framework with React.js. A database system PostgreSQL is used to store user data, blog posts, and media content efficiently. The required tools and development environments are easily accessible and do not demand high-end hardware.

The system can also be implemented with standard features like user authentication, content management, and multimedia integration without major technical challenges. Additionally, the platform can be designed to be responsive and compatible across different devices and browsers, ensuring smooth performance. Therefore, the project is technically achievable with the current knowledge and resources available.

Operational Feasibility:

The Band Blogging Website is operationally feasible because it is designed to be simple and user-friendly for both bands and users. Band members can easily create, edit, and manage their blog posts without requiring technical expertise, while users can browse and interact with content effortlessly.

The system does not require special training, as the interface is intuitive and similar to commonly used websites. It can be smoothly integrated into regular use, allowing bands to update content regularly and users to access information anytime. Additionally, basic maintenance and management can be handled without difficulty.

Overall, the system fits well within normal operations and can be effectively used by its intended users without major issues.

Economic Feasibility:

The Band Blogging Website is economically feasible because it requires minimal investment for development and maintenance. The project can be built using low-cost technologies such as open-source frameworks, databases, and development tools. There is no need for expensive hardware or software, which helps reduce overall costs.

Additionally, hosting and deployment can be done using affordable platforms, making it budget-friendly. The benefits of the system, such as improved communication, better promotion for bands, and enhanced user engagement, outweigh the development costs.

Overall, the project is cost-effective and provides good value, making it economically feasible.

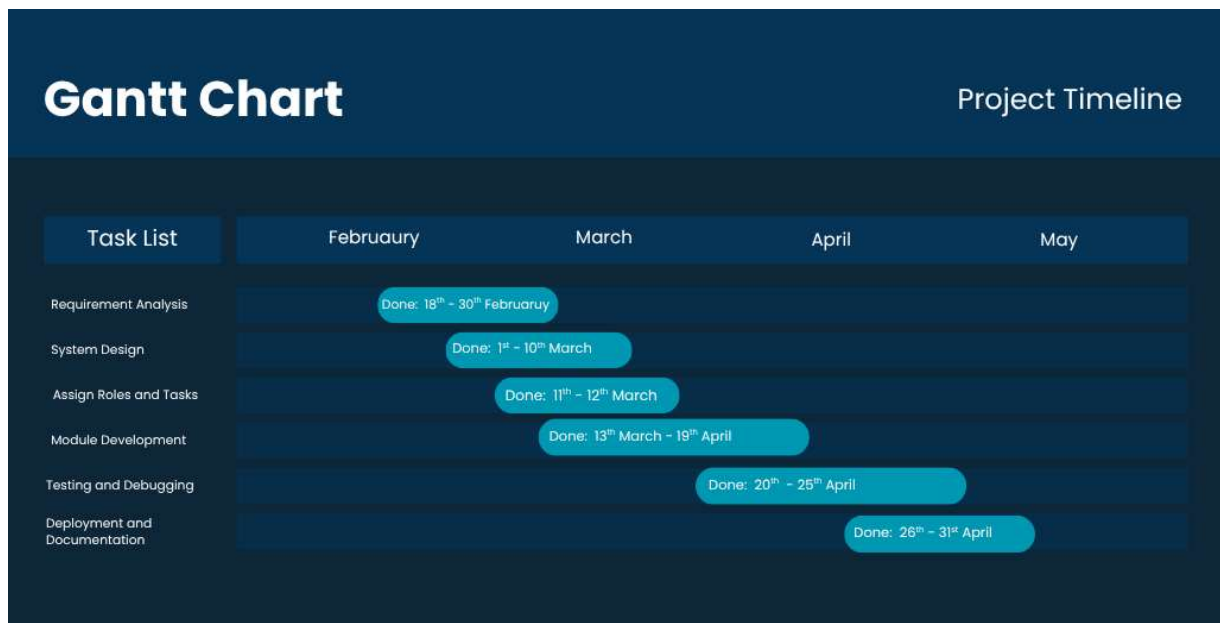
3.4 PROJECT PLANNING

An authorized document that acts as the project vision and describes the project's execution, as well as how it will be evaluated & regulated. It displays all details of the steps that will be implemented in a project, such as the timing and resources that will be used to complete the project. It may be quite detailed, which typically results in a better plan, it can also include secondary plans other planning papers.

3.5 PROJECT SCHEDULING

A project schedule is simply a method for conveying and understanding what activities are required & must be accomplished, what organizational resources will be provided or allocated to these tasks, and when the work must be completed. Cost savings, Successful project, & customer pleasure all benefit from effective project scheduling.

GANTT CHART:



1. Requirement Analysis (18th – 30th February)

- The project began with a detailed requirement analysis phase.
- The team identified the core objectives: blog creation and management, user authentication, multimedia integration, and user interaction features (comments/likes).
- User needs and system feasibility were evaluated.
- This phase ensured a clear understanding of functional and non-functional requirements.

2. System Design (1st – 10th March)

- The overall architecture of the Band Blogging Website was designed.
- Key modules such as user management, blog posting system, database structure, and UI flow were planned.
- The design ensured scalability and smooth integration between the frontend, backend, and database.

3. Assign Roles and Tasks (11th – 12th March)

- Team members were assigned specific responsibilities.
- Tasks were divided into modules such as UI development, backend development, database management, and testing.
- This ensured efficient workflow and parallel development.

4. Module Development (13th March – 19th April)

- Each system component was developed based on the finalized design.
- Features for blog posting, editing, media uploads, and user interaction were implemented.
- Frontend interfaces and backend functionalities were created and connected with the database.

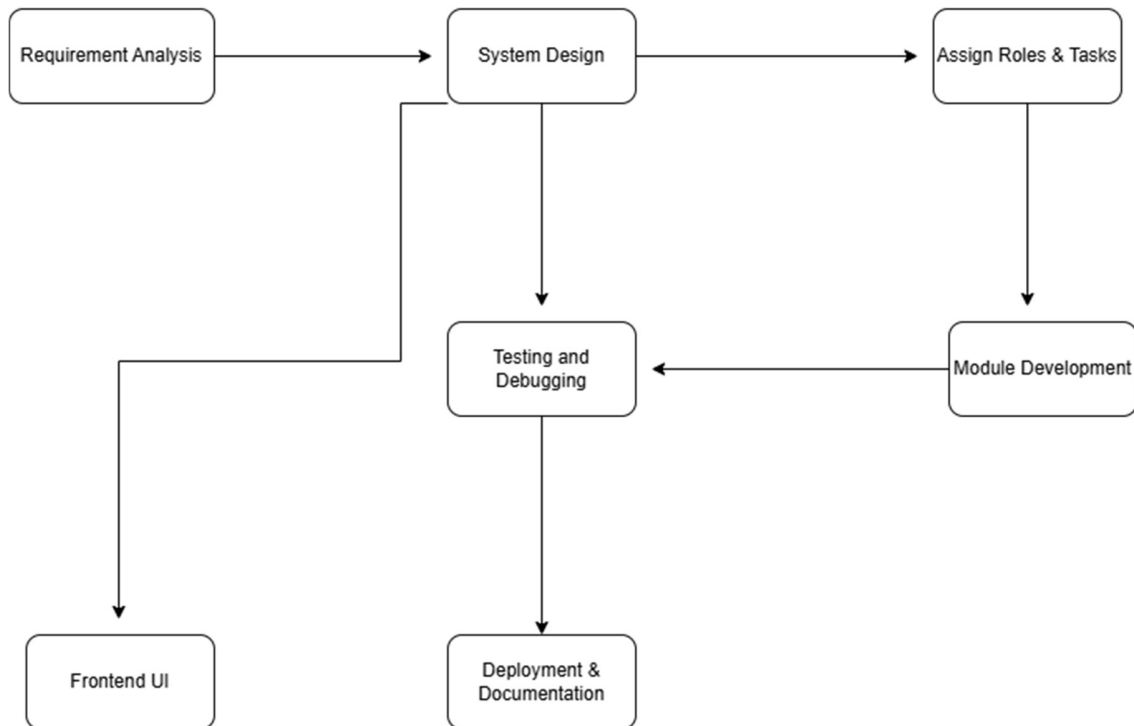
5. Testing and Debugging (20th – 25th April)

- The entire website underwent systematic testing.
- Bugs in blog posting, user interaction, and UI performance were identified and fixed.
- Final improvements were made to ensure smooth functionality and stability.

6. Deployment and Documentation (26th – 31st April)

- The completed website was deployed in its final environment.
- User guides, technical documentation, and project reports were prepared.
- The system was finalized for presentation and evaluation.

PERT CHART



1. Requirement Analysis

- This is the starting point of the project where all system requirements were gathered.
- It includes identifying user needs, understanding project scope, and defining core functionalities.
- It serves as the foundation for all subsequent tasks.

2. System Design

- Conducted after requirement analysis, this phase focuses on planning the system architecture.
- It includes designing the frontend layout, backend workflow, and database structure.
- Ensures all modules will integrate smoothly.

3. Assign Roles and Tasks

- Responsibilities are distributed among team members.
- Tasks are divided into modules such as frontend, backend, and processing components.
- Helps in efficient and parallel development.

4. Module Development

- **Backend Integration**

- Connects all modules and manages data flow.
- Ensures communication between UI and database.

- **Frontend UI**

- Provides a user-friendly interface.
- Allows users to check on what's something new happened on the website.

5. Testing and Evaluation

- All modules are tested together for errors and performance issues.
- Ensures accuracy and system stability.
- Necessary improvements are made.

6. Frontend UI

- The frontend module is responsible for building a user-friendly interface.
- Users can read news, view upcoming events and previous, and check on newly released song.
- It ensures easy access for remote fans.

7. Deployment and Documentation

- The final system is deployed after successful testing.
- Includes server setup and documentation preparation.
- Marks the completion of the project.

3.6 SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

The Band Blogging Website is designed to provide an interactive platform where music bands can share blogs, updates, and multimedia content with their audience. The system will support two main types of users: administrators (band members) and general users (visitors). Administrators will be able to create, edit, and delete blog posts, upload images or videos, and manage user interactions. General users will be able to browse blogs, view content, and engage through comments or feedback features.

The functional requirements of the system include user registration and login, blog management, media upload, and comment functionality. The system should allow bands to organize content efficiently and ensure that users can easily navigate and access information. Non-functional requirements include performance, usability, security, and scalability. The website should load quickly, provide a user-friendly interface, and ensure secure handling of user data through authentication mechanisms.

The system will be developed using modern web technologies such as HTML, CSS, JavaScript, and a backend framework with database support. It should be compatible with multiple devices and browsers to ensure accessibility. Overall, the Band Blogging Website aims to create a reliable, efficient, and engaging platform that enhances communication between bands and their audience while supporting future expansion and feature integration.

3.7 SOFTWARE ENGINEERING PARADIGM APPLIED

The Band Blogging Website is developed using the **Waterfall Model**, a linear and sequential software engineering paradigm. This approach was chosen because the project requirements were clearly defined at the beginning, allowing the development process to follow a structured path. Each phase of the project is completed before moving on to the next, ensuring clarity, proper documentation, and systematic progress.

The development process began with the requirement analysis phase, where the needs of bands and users were identified. This was followed by the system design phase, in which the overall architecture, including frontend, backend, and database structure, was planned. After finalizing the

design, the implementation phase was carried out, where different modules such as user authentication, blog management, and user interface were developed.

Once the development was completed, the project moved into the testing phase. During this phase, all components were integrated and tested to identify and fix errors, ensuring that the system functions correctly and efficiently. Finally, the deployment phase involved launching the website and preparing documentation for users and developers.

The Waterfall Model is suitable for this project as it emphasizes proper planning, documentation, and a clear sequence of steps. It helps in maintaining discipline throughout the development process and ensures that each stage is thoroughly completed before proceeding. Overall, this paradigm supports the successful and organized development of the Band Blogging Website.

3.8 DATA MODELS

Level 0: Context Diagram

At this level, the diagram shows the system as a single process and its interaction with external entities.



Data Flow:

1. User Interaction:

The user accesses the Band Blogging Website to browse blogs, view multimedia content, and interact through comments or feedback. Users can explore posts published by different bands and stay updated with their latest activities.

2. Processing by Band Blogging Website:

The Band Blogging Website processes user requests and displays relevant content. It manages blog posts, handles user interactions, and ensures smooth navigation between different sections of the website.

3. System Access by Admin:

The admin (band members) accesses the system to create, edit, and delete blog posts. They can upload images, videos, and update information related to their band activities.

4. Admin Controls and Management:

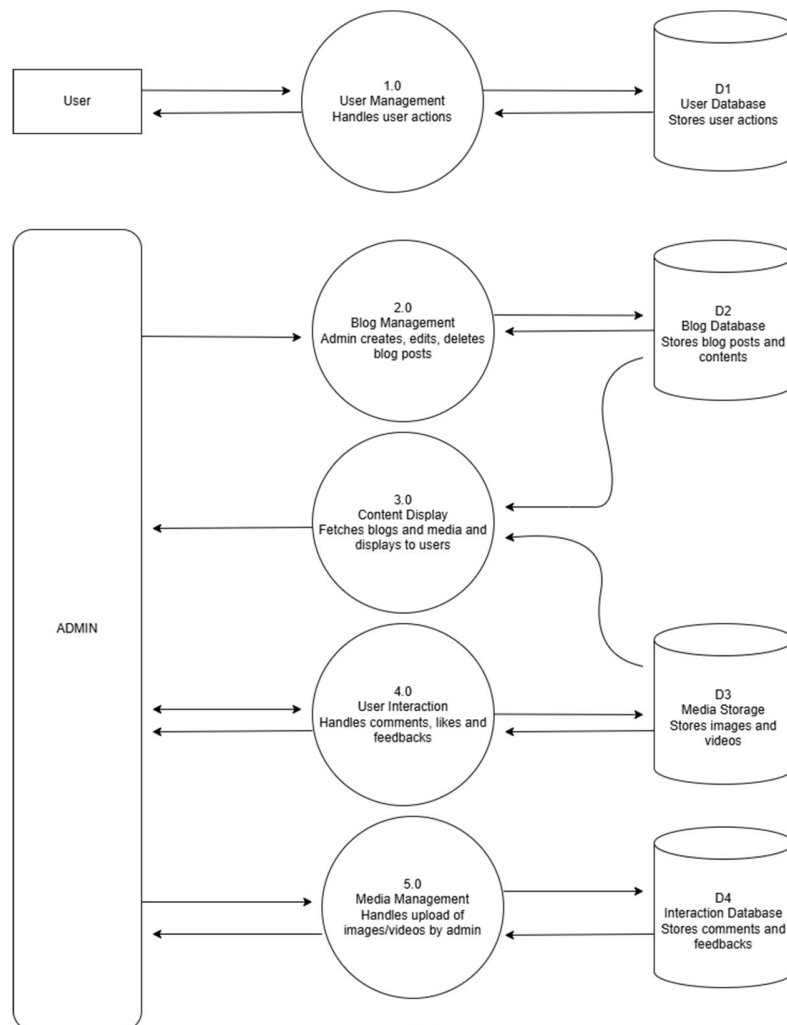
The admin manages the overall content of the website, including moderating comments, updating posts, and maintaining the quality and relevance of information shared on the platform.

5. Output to User:

The Band Blogging Website delivers the final output to the user in the form of displayed blogs, media content, and updates. Users can view, read, and interact with the content seamlessly.

Level 1: Detailed DFD

This level breaks down the Band Blogging Website into its core modules and shows how data flows between them.



1. User to User Management (1.0):

The user interacts with the system by registering or logging in through the User Management module. This module validates user credentials and processes user actions.

2. User Management to User Database (D1):

The User Management module stores and retrieves user information such as login details and profile data from the User Database.

3. Admin to Blog Management (2.0):

The admin (band member) creates, edits, or deletes blog posts using the Blog Management module. This module controls all blog-related operations.

4. Blog Management to Blog Database (D2):

All blog content, including text and updates, is stored and retrieved from the Blog Database for further use.

5. Content Display (3.0) to User:

The Content Display module fetches blog posts and media from the databases and presents them to the user in an organized and readable format.

6. User to User Interaction (4.0):

Users interact with the content by adding comments, likes, or feedback through the User Interaction module.

7. User Interaction to Interaction Database (D4):

All user interactions such as comments and feedback are stored in the Interaction Database and can be retrieved when needed.

8. Admin to Media Management (5.0):

The admin uploads images and videos using the Media Management module, which handles all media-related operations.

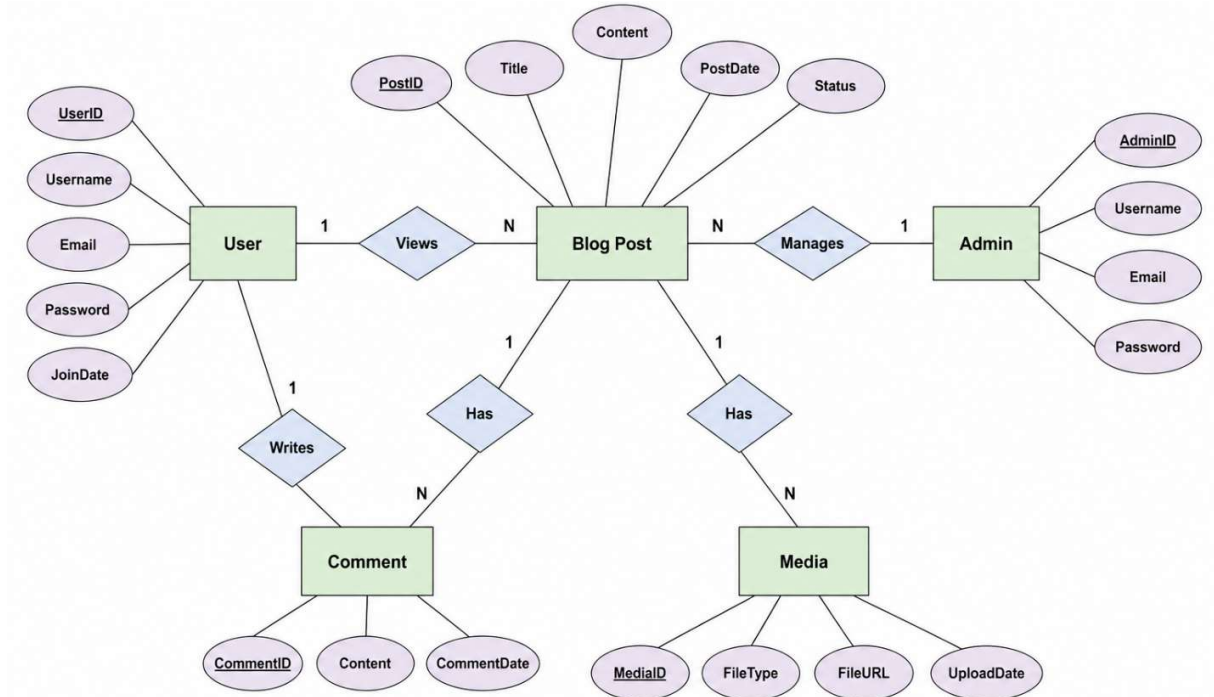
9. Media Management to Media Storage (D3):

Uploaded media files are stored in the Media Storage and are retrieved when required for display.

10. Final Output to User:

The system delivers the final output to the user in the form of blogs, images, videos, and interactive content, ensuring a smooth and engaging experience.

Entity Relationship Diagram:



The Entity–Relationship (ER) Diagram visually represents the logical structure of the database used in the Band Blogging Website. The system stores and manages data related to users, blog posts, comments, and media. The ER diagram helps illustrate how these entities are connected and how data flows within the system.

1. USER Entity

The USER entity represents visitors who interact with the website.

It includes attributes such as:

- UserID – Unique identifier for each user
- Username – Name of the user
- Email – User’s email address
- Password – Login credential
- JoinDate – Date when the user registered

A user can view multiple blog posts and write comments.

2. BLOG POST Entity

The BLOG POST entity stores information about blog content created by the admin (band).

It includes attributes such as:

- PostID – Unique identifier for each blog
- Title – Title of the blog post
- Content – Main blog content
- PostDate – Date of publication
- Status – Published or draft status

Each blog post is managed by an admin and can have multiple comments and media files.

3. ADMIN Entity

The ADMIN entity represents band members who manage the website.

It includes:

- AdminID – Unique identifier
- Username – Admin name
- Email – Contact details
- Password – Login credential

An admin can manage multiple blog posts.

4. COMMENT Entity

The COMMENT entity stores user feedback on blog posts.

Attributes include:

- CommentID – Unique identifier
- Content – Comment text
- CommentDate – Date of comment

Each comment is written by a user and belongs to one blog post.

5. MEDIA Entity

The MEDIA entity stores images and videos related to blog posts.

Attributes include:

- MediaID – Unique identifier
- FileType – Type of media (image/video)
- FileURL – Location of file
- UploadDate – Date uploaded

Each media file is associated with a blog post.

6. Relationships

- **A User views Blog Posts (1:N)**
- **A User writes Comments (1:N)**
- **A Blog Post has Comments (1:N)**
- **A Blog Post has Media (1:N)**
- **An Admin manages Blog Posts (1:N)**

Summary

The ER diagram clearly shows that users interact with blog posts by viewing and commenting, while admins manage the content. Blog posts act as the central entity connecting comments and media. This structure ensures organized data storage, efficient retrieval, and scalability for the Band Blogging Website.

4. SYSTEM DESIGN

The System Design for the **Band Blogging Website** focuses on a well-structured and modular approach to ensure the system is maintainable, scalable, and efficient. The system is divided into clearly defined modules such as user management, blog management, media handling, and user interaction. It follows a separation of concerns by distinguishing between the frontend (user interface for browsing and interaction) and the backend (handling data processing, authentication, and database operations), enabling smooth functionality and easier future enhancements.

4.1 MODULARIZATION

Modularization plays a crucial role in the design and development of the **Band Blogging Website**. By dividing the system into smaller, manageable, and independent modules, the project becomes easier to develop, test, maintain, and enhance in the future. Each module performs a specific function and interacts with others through well-defined data flows. This structured approach reduces complexity, improves clarity, and allows team members to work on different parts of the system simultaneously without integration issues.

The User Management Module handles user registration, login, and authentication, ensuring secure access to the system. The Blog Management Module allows admins (band members) to create, edit, and delete blog posts, managing the core content of the website. The Content Display Module is responsible for fetching and presenting blogs and media to users in an organized and user-friendly manner.

The User Interaction Module enables users to engage with content by adding comments, likes, and feedback. The Media Management Module allows admins to upload and manage images and videos associated with blog posts, ensuring rich and engaging content for users.

The Database Module supports all other modules by storing user data, blog posts, comments, and media files. It ensures efficient data retrieval and maintains consistency across the system.

Finally, the User Interface Module provides a smooth and interactive experience, allowing users to browse content and interact easily. Through modularization, the Band Blogging Website achieves better organization, scalability, and maintainability, making it adaptable for future improvements.

Creating a Repository in GitHub

This section provides a comprehensive, step-by-step explanation of how to create a new repository on GitHub, based on the interface shown in the provided screenshot

1. Step 1:

```
PS C:\Users\NEW> echo "# band-website" >> README.md
```

- Creates a line of text # band-website
- Appends (>>) it into a file named README.md
- README.md is usually the main description file of a project

2. Step 2:

```
PS C:\Users\NEW> git init  
Initialized empty Git repository in C:/Users/NEW/.git/
```

- Initializes a new Git repository in the current folder
- Creates a hidden .git directory
- Starts version control for the project

3. Step 3:

```
PS C:\Users\NEW> git add README.md
```

- Adds README.md to the staging area
- Git now tracks this file and prepares it for commit.

4. Step 4:

```
PS C:\Users\NEW> git commit -m "first commit"  
[master (root-commit) da76e4e] first commit  
1 file changed, 0 insertions(+), 0 deletions(-)  
create mode 100644 README.md
```


- Saves the staged changes permanently in Git history
- -m adds a commit message

5. Step 5:

```
PS C:\Users\NEW> git remote add origin https://github.com/joji-equ/band-website.git
```

- Renames the current branch to main
- -M forces the rename even if a branch already exists

Modern GitHub repositories usually use main as the default branch.

6. Step 6:

```
PS C:\Users\NEW> git remote add origin https://github.com/joji-equ/band-website.git
```

- Connects the local repository to a remote GitHub repository
- origin is the nickname for the GitHub repo URL

7. Step 7:

```
PS C:\Users\NEW> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 239 bytes | 239.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/joji-equ/band-website.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- Uploads (pushes) your local main branch to GitHub
- -u sets origin/main as the default upstream branch

4.2 DATA INTEGRITY AND CONSTRAINTS

Data Integrity and Constraints

Data integrity is one of the most critical aspects of the Band Blogging Website, as it ensures the accuracy, consistency, and reliability of all the information stored and processed within the system. Since the project involves multiple modules such as user management, blog management, media handling, user interaction, and database storage, maintaining strong data integrity is essential to ensure that blog content, user details, comments, and media files remain accurate and consistent. It plays a key role in providing a trustworthy platform where users can access correct and up-to-date information shared by bands. Data integrity is especially important in managing user authentication, preventing duplicate or invalid entries, and ensuring that all relationships between entities (such as users, blogs, comments, and media) are properly maintained. Therefore, various validation techniques, constraints, and integrity rules are implemented throughout the system to protect data quality and ensure smooth and reliable operation of the website.

1. Input Data Integrity

The integrity process begins with user and admin inputs. All data entered into the system must follow specific constraints before being accepted. These include:

- Input validation: Ensuring correct formats for usernames, emails, and passwords.
- Content validation: Blog posts must follow proper structure and avoid invalid or harmful content.
- Media validation: Only supported file formats (images/videos) are allowed.
- User authentication: Ensures that only authorized users and admins can access or modify data.

These measures ensure that all incoming data is accurate, safe, and suitable for processing.

2. User Data Integrity

For user-related operations, the system maintains:

- Unique user identification: Each user has a unique ID and email.
- Secure data handling: Passwords are stored securely.
- Validation checks: Prevent duplicate accounts or invalid entries.

This ensures reliable and consistent user information.

3. Blog Content Integrity

During blog creation and management, the system enforces:

- Content validation: Ensures blogs contain meaningful and relevant information.
- Format consistency: Titles, descriptions, and content follow a proper structure.
- Update control: Only authorized admins can edit or delete posts.

These rules maintain the quality and authenticity of blog content.

4. User Interaction Integrity

For comments and feedback:

- Spam prevention: Filters to block inappropriate or duplicate comments.
- Relationship validation: Each comment must be linked to a valid user and blog post.
- Content moderation: Admins can review and manage user interactions.

This ensures a safe and meaningful interaction environment.

5. Database Integrity Constraints

The database ensures long-term consistency through:

- Primary and foreign keys: Maintain relationships between users, blogs, comments, and media.
- Normalization: Prevents data redundancy.
- Referential integrity: Ensures all records are properly linked.
- Access control: Restricts unauthorized modifications.

These mechanisms prevent data inconsistency and loss.

6. Output Integrity

When content is displayed to users:

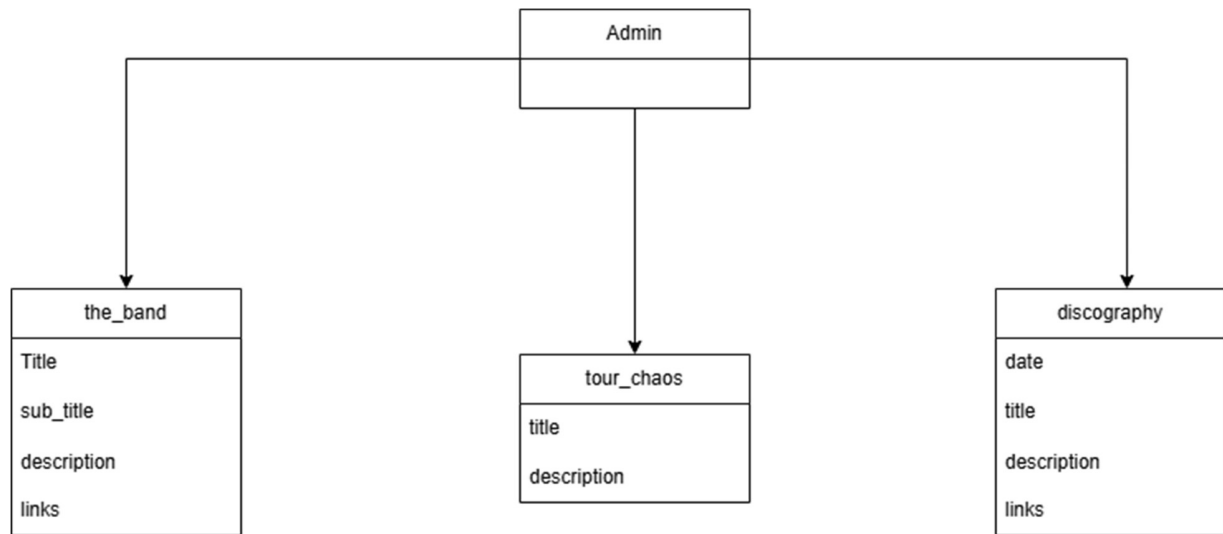
- Accurate data retrieval: Ensures correct blogs, media, and comments are shown.
- Complete information: No missing or broken content.
- Proper formatting: Ensures a clean and readable interface.

Conclusion

By applying these data integrity rules across all stages from input validation to database management—the Band Blogging Website ensures reliable, consistent, and high-quality **content**.

These measures help create a secure and trustworthy platform for both bands and users.

4.3 DATABASE DESIGN



The database design for the **Band Blogging Website** is simple and focused on managing blog-related content rather than complex user systems. It primarily handles information about admins, band details, tours, and discography. The design ensures that content is well-organized, easy to manage, and efficiently stored. The diagram consists of key entities such as Admin, Band, Tour, and Discography, which work together to support the website's functionality.

1. Admin Entity

The Admin entity represents the band members or authorized users who manage the website.

- Admin is responsible for adding, updating, and managing all content on the platform
- It acts as the controlling entity for other data components
- Ensures only authorized access to modify content

2. Band (the_band) Entity

This entity stores information related to the band.

- Includes details such as band name, description, and background information
- Represents the core identity of the website
- Managed and updated by the admin

3. Tour (tour_chaos) Entity

The Tour entity contains information about events, concerts, or tours organized by the band.

- Stores tour dates, locations, and event details
- Helps users stay updated with upcoming performances
- Linked conceptually to the band

4. Discography Entity

The Discography entity stores details about the band's music releases.

- Includes albums, songs, and release information
- Allows users to explore the band's musical work
- Maintained by the admin

5. Relationships Between Entities

- The **Admin manages** all entities (Band, Tour, Discography)
- The **Band entity connects** with Tour and Discography as it represents the main subject
- Tours and Discography are dependent on the band's information

Summary

The database design is lightweight and content-focused, ensuring efficient storage and easy management of band-related information. It avoids unnecessary complexity while maintaining clear relationships between entities. This structure supports smooth content updates and scalability for the Band Blogging Website.

4.4 USER INTERFACE

The user interface (UI) of the Band Blogging Website is designed to be clean, modern, and user-friendly, allowing visitors to easily navigate through the platform and explore band-related content. The layout follows a visually appealing design with well-structured sections, clear typography, and intuitive navigation. All key features such as blogs, band information, tours, and discography are easily accessible to enhance user experience.

1. Navigation Bar (Header Section)

At the top of the interface, a navigation bar is provided for easy access to different sections of the website.

- Includes links such as Home, Band, Tours, Discography, and Contact
- Helps users quickly move between pages
- Maintains a consistent layout across the website

2. Main Header / Hero Section

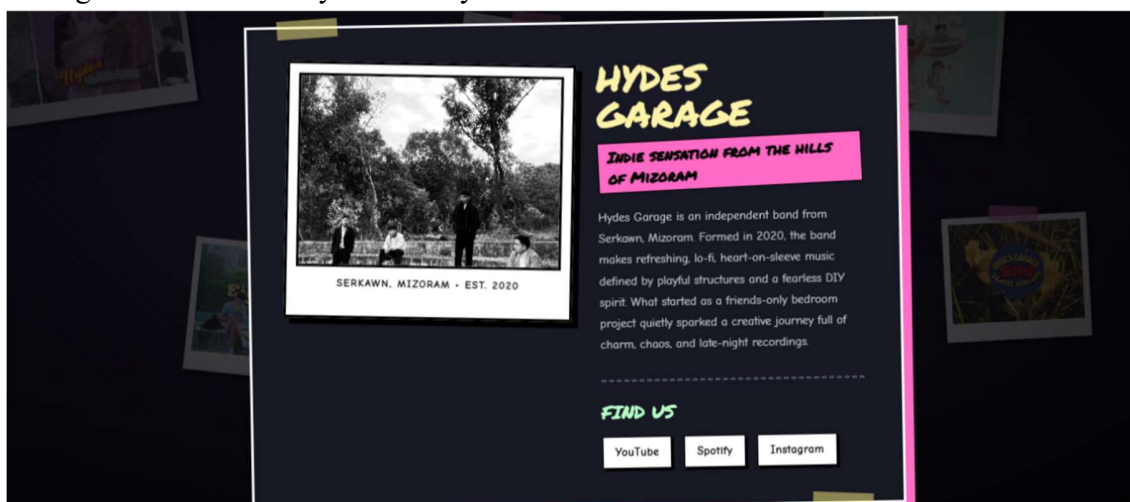
The homepage features a prominent header section that highlights the identity of the band.

- Displays the band name and tagline clearly
- Uses attractive visuals or background images
- Creates a strong first impression for visitors

3. Band Information Section

This section provides details about the band.

- Includes background, history, and description
- Helps users understand the band's identity and journey
- Well-organized text for easy readability



4. Tours Section

The Tours section displays upcoming and past events.

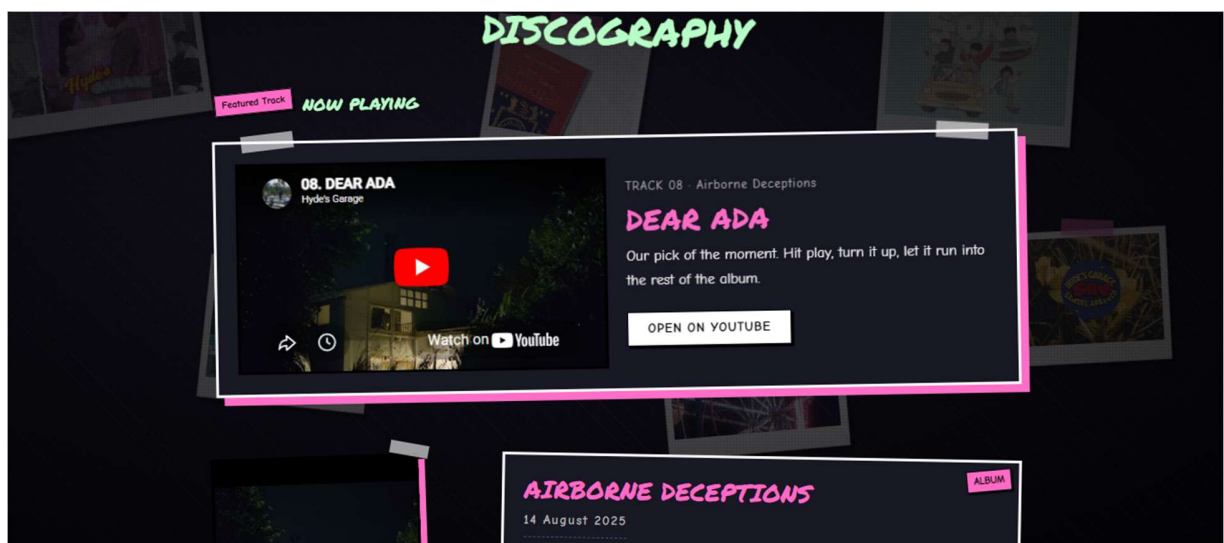
- Shows event dates, locations, and details
- Keeps fans updated about performances
- Structured in a clear and readable format



5. Discography Section

This section showcases the band's music releases.

- Includes albums, songs, and release details
- Allows users to explore the band's work
- Presented in an organized and visually appealing way



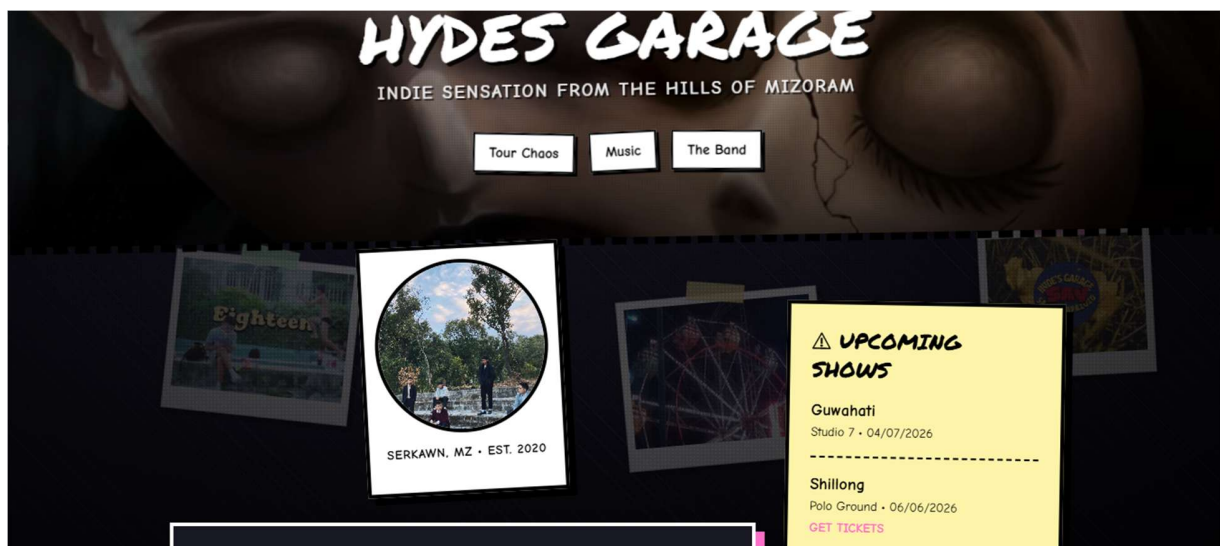
6. User Interaction / Contact Section

Users can interact with the website through the contact or feedback section.

- Allows users to send messages or inquiries
- Enhances communication between fans and the band

7. Overall Design Features

- Responsive layout for different devices (mobile, tablet, desktop)
- Simple and intuitive navigation
- Clean design with consistent color theme
- Focus on content readability and user engagement

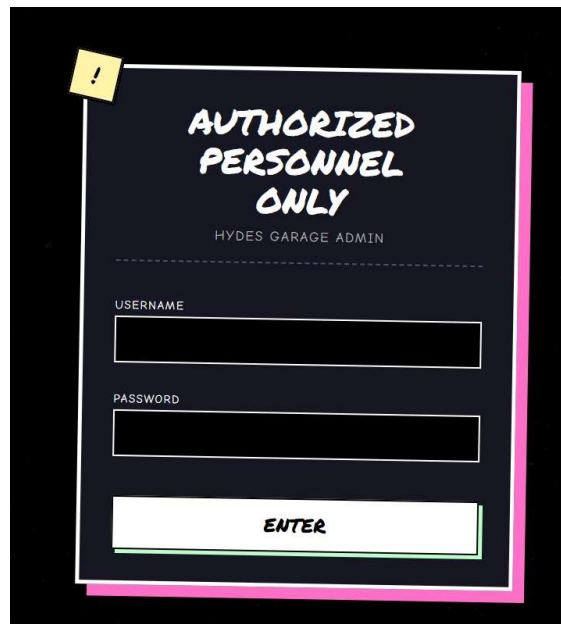


This UI design ensures that users can easily explore the Band Blogging Website, stay updated with the band's activities, and enjoy a smooth and engaging browsing experience.

Admin Panel:

1. Login page:

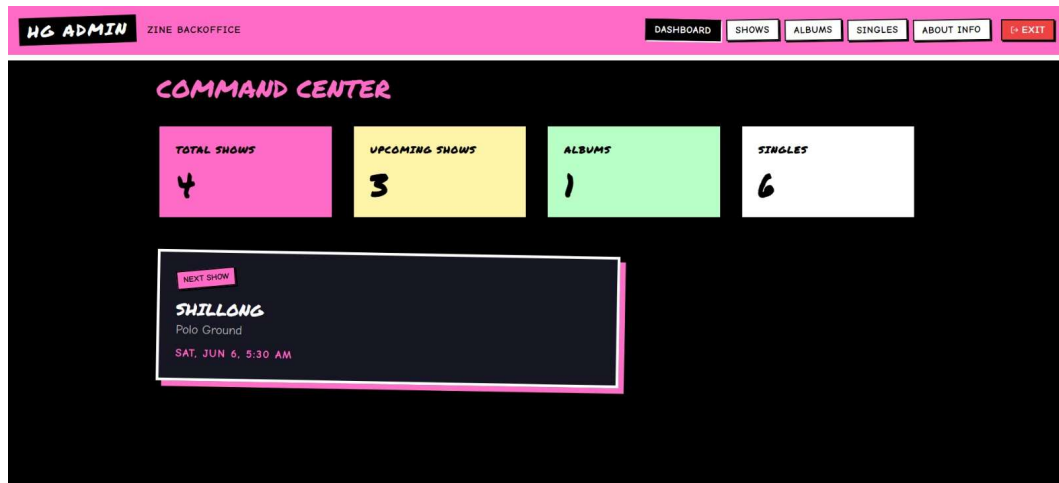
The Admin Login Page is designed to provide secure access to authorized personnel responsible for managing the Band Blogging Website. This page ensures that only admins can access the administrative dashboard and perform operations such as managing blog posts, updating tour information, uploading media, and maintaining website content.

A mockup of an admin login page. It features a dark blue background with a white border. At the top, the text "AUTHORIZED PERSONNEL ONLY" is written in a bold, white, sans-serif font. Below this, "HYDES GARAGE ADMIN" is written in a smaller, white, sans-serif font. A dashed line separates the header from the login fields. There are two input fields: "USERNAME" and "PASSWORD", both with white outlines. Below the password field is a white button with the text "ENTER" in a bold, black, sans-serif font. A yellow sticky note with an exclamation mark is attached to the top left corner of the form.

2. Admin Dashboard:

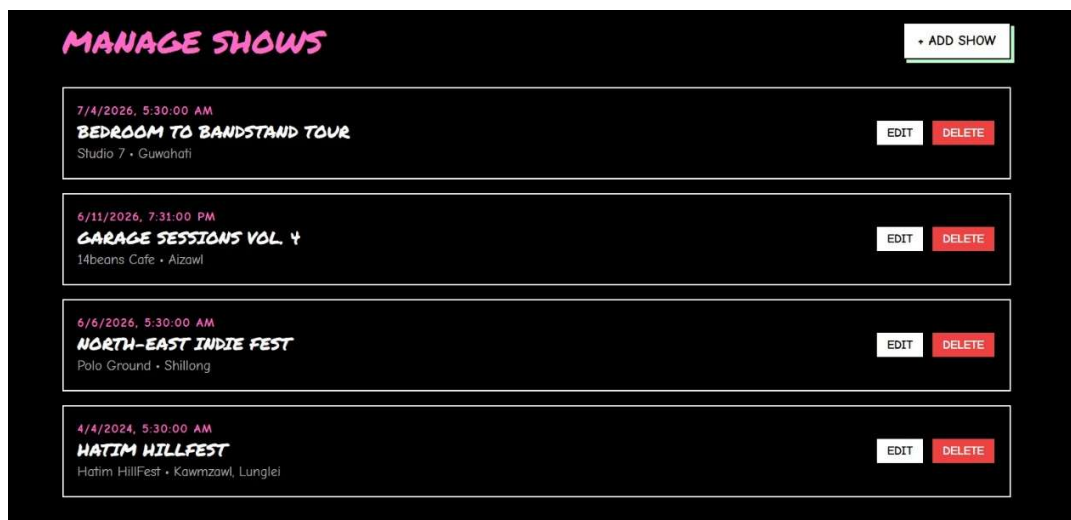
The Admin Dashboard Page serves as the central control panel of the Band Blogging Website. It allows authorized admins to manage website content efficiently, including shows, albums, singles, and band information. The dashboard provides a quick overview of important statistics and upcoming events, helping admins monitor and update the platform easily.

The top navigation bar contains different management sections such as **Dashboard**, **Shows**, **Albums**, **Singles**, and **About Info**, along with an **Exit** button for secure logout. This navigation structure enables admins to quickly switch between different administrative functions.



3. Manage Shows Panel:

The Manage Shows Page allows administrators to efficiently manage all event and tour information for the Band Blogging Website. This section enables admins to add, update, and remove show details, ensuring that fans always receive accurate and up-to-date event information. At the top of the page, an “+ Add Show” button is provided, allowing admins to create and publish new show or tour entries. This feature helps keep the website updated with the latest performances and upcoming events.



4. Manage Albums Page:

The Manage Albums Page allows administrators to manage and organize album information on the Band Blogging Website. This module helps admins maintain updated records of the band's music releases, ensuring fans can easily explore the band's discography.

At the top section of the page, an “+ Add Album” button is provided. This feature enables admins to add new albums along with their release details to the system. It simplifies the process of updating the website whenever a new album is released.



5. Manage Singles Page:

The Manage Singles Page is designed to help administrators manage individual music releases on the Band Blogging Website. This section allows admins to add, update, and remove singles, ensuring that the website always displays the latest music content for fans and visitors.

At the top of the page, an “+ Add Single” button is provided, enabling admins to easily upload and publish new singles. This feature helps keep the music section updated whenever the band releases a new track.



6. Manage Info/About Panel/Page:

The Manage Info/About Page allows administrators to manage the informational content of the Band Blogging Website. This section is used to update details related to the band's background, history, achievements, and overall identity presented to visitors.

The page provides a simple and organized interface where admins can edit and maintain the "About" section of the website. This ensures that users always receive accurate and updated information about the band and its activities.

The screenshot shows a web form titled "EDIT ABOUT INFO" in a yellow, stylized font. The form is set against a dark background and contains several input fields and a large text area. The fields are organized into two columns. The first column includes "Band Name" (filled with "Hydes Garage"), "Hometown" (filled with "Serkawn, Mizoram"), "Photo URL" (empty), and "Spotify URL" (filled with a Spotify link). The second column includes "Tagline" (filled with "Indie sensation from the hills of Mizoram"), "Founded Year" (filled with "2020"), "YouTube URL" (filled with a YouTube channel link), and "Instagram URL" (filled with an Instagram profile link). A large text area labeled "Story / Bio" contains a paragraph about the band. At the bottom, a yellow button is labeled "SAVE ABOUT INFO".

EDIT ABOUT INFO

Band Name	Hydes Garage	Tagline	Indie sensation from the hills of Mizoram
Hometown	Serkawn, Mizoram	Founded Year	2020
Story / Bio			
Hydes Garage is an independent band from Serkawn, Mizoram. Formed in 2020, the band makes refreshing, lo-fi, heart-on-sleeve music defined by playful structures and a fearless DIY spirit. What started as a friends-only bedroom project quietly sparked a creative journey full of charm, chaos, and late-night recordings.			
Photo URL		YouTube URL	
		https://www.youtube.com/@hydesgarage	
Spotify URL		Instagram URL	
https://open.spotify.com/artist/44qrg8E3Et3bLqNzkazN		https://www.instagram.com/hydesgarage?igsh=MTg2OXBhODF2ZGV4cw==	
SAVE ABOUT INFO			

5. TESTING

Testing is an essential phase in the development of the Band Blogging Website, ensuring that all modules function as expected, data flows correctly across components, and the system meets the functional and non-functional requirements. This section outlines the testing techniques, strategies used, and sample test reports covering both unit testing and system testing.

5.1 Testing Techniques and Testing Strategies Used

Testing Techniques

1. Black Box Testing:

This technique was used to test the system's functionalities without examining internal code. Modules such as User Management, Blog Management, Media Handling, and User Interaction were tested using different inputs to verify expected outputs. This ensured correct system behavior under normal and edge cases.

2. White Box Testing:

This was applied during development to validate internal logic of the system. It focused on checking conditions, loops, authentication logic, and data handling processes within modules like login validation, blog operations, and database interactions.

3. Unit Testing:

Each module was tested individually to ensure correctness. For example, user login, blog creation, media upload, and comment features were tested separately to identify and fix errors at an early stage.

4. Integration Testing:

After unit testing, different modules were combined and tested together. This ensured proper communication between components such as User Management → Blog Management → Database and Media Management → Content Display.

5. System Testing:

The complete Band Blogging Website was tested as a whole to verify that it meets all system requirements. This included testing functionality, performance, responsiveness, and overall usability.

6. User Acceptance Testing (UAT):

A group of users tested the website to evaluate usability, navigation, and overall experience. Their feedback helped improve the interface, content display, and interaction features.

5.2 Test Reports for Unit Case and System Test Cases

A. Unit Test Case Report

Test Case ID	Module	Input	Expected Outcome	Actual Output	Status
UT-01	Admin Management	Valid admin login credentials	Admin successfully logs in	Admin logged in successfully	Pass
UT-02	Admin Management	Invalid login credentials	Error message displayed	Error message displayed	Pass
UT-03	Blog Management	Valid blog post data	Blog post created successfully	Blog created successfully	Pass
UT-04	Blog Management	Edit existing blog	Blog updated successfully	Blog updated successfully	Pass
UT-05	Media Management	Upload valid image/video	File uploaded successfully	File uploaded successfully	Pass
UT-06	Database Module	Store blog data	Data saved with proper ID and relation	Data saved successfully	Pass

These unit tests ensured that each building block of the system worked independently without error.

B. System Test Case Report

Test Case ID	Scenario Description	Input	Expected Outcome	Actual Output	Status
ST-01	Admin login and access system	Valid login credentials	Admin successfully logs in and accesses dashboard	Login successful	Pass
ST-02	Create and publish blog	Valid blog content	Blog is created and displayed to users	Blog displayed correctly	Pass
ST-03	Handle invalid blog input	Empty/invalid blog fields	Error message displayed	Error displayed	Pass
ST-04	Upload media files	Valid image/video file	Media uploaded and linked to blog	Media uploaded successfully	Pass
ST-05	View blog content	User selects a blog	Blog content, media, and details displayed properly	Content displayed correctly	Pass
ST-06	Full system workflow	Login → Create Blog → Upload Media → View	Complete flow works smoothly without errors	Workflow successful	Pass

System testing validated the functional completeness, workflow stability, and output correctness of the entire application.

Summary

The combination of black-box, white-box, unit, integration, and system testing ensured that the Band Blogging Website operates efficiently and accurately. All core modules passed their tests, confirming system readiness for real-world usage.

6. SYSTEM SECURITY MEASURES

System security is a critical aspect of the Band Blogging Website, ensuring that user data, blog content, media files, and database records are protected from unauthorized access or misuse. Since the system involves user interactions and content management by admins (band members), strong security measures are implemented across different layers of the system.

The first layer of security focuses on authentication and access control. Only authorized admins can create, edit, or delete blog posts and upload media files. User credentials are securely handled, and passwords are stored using encryption or hashing techniques to prevent unauthorized access. Session management is also implemented to maintain secure user interactions and prevent misuse.

The Media Management Module includes validation mechanisms to ensure that only supported file formats (such as images and videos) are uploaded. File size limits and server-side checks are applied to prevent malicious uploads and system overload. This helps protect the system from harmful files and ensures smooth operation.

Communication between the frontend and backend is secured using HTTPS protocols during deployment, ensuring that all data exchanged is encrypted. This prevents data interception and protects user information from external threats.

At the database level, integrity constraints, access control rules, and secure queries are implemented to prevent unauthorized modifications and attacks such as SQL injection. Only authorized modules can access or modify sensitive data, reducing security risks.

Additionally, the system includes proper error handling and logging mechanisms to monitor unusual activities without exposing sensitive information. Regular backups are maintained to ensure data recovery in case of system failure.

Overall, these security measures ensure the confidentiality, integrity, and availability of data, making the Band Blogging Website a secure and reliable platform for both users and administrators.

7. COST ESTIMATION

The cost estimation for the **Band Blogging Website** project is based on the tools, technologies, and platforms used during development. Since the project primarily relies on free and open-source technologies, the overall development cost is minimal. However, identifying potential cost components is important for future scalability and deployment planning.

The system is developed using standard web technologies such as HTML, CSS, JavaScript, and backend framework with React.js, which are freely available. For database management, PostgreSQL is used under their free-tier plans, allowing efficient data storage without additional cost. These tools provide sufficient features for development and testing without requiring paid subscriptions.

For hosting and deployment, free and low-cost platforms Replit is used. These platforms offer basic hosting, bandwidth, and uptime management at no cost during the initial stages of the project. This significantly reduces expenses related to server infrastructure.

All development activities were carried out using personal computers and existing internet connections, resulting in no extra hardware or network costs. Additionally, commonly used development tools and editors are freely available, further minimizing expenditure.

Although the project does not involve direct spending, documenting cost estimation highlights its economic feasibility. The use of free-tier services and open-source tools makes the Band Blogging Website highly cost-effective, making it suitable for students and small-scale developers while still allowing future expansion if required.

8. REPORTS

The reporting component of the **Band Blogging Website** project plays an important role in documenting the development process, system functionality, performance, and final outcomes. Reports ensure clarity, transparency, and proper communication for instructors, evaluators, and future developers who may enhance the system. Throughout the development lifecycle, different reports were prepared to cover each phase, including analysis, design, implementation, and testing.

The **Requirements Report** documents both functional and non-functional requirements identified during the system analysis phase. It defines the system objectives, user needs, features such as blog management and user interaction, and the overall scope of the project. This report acts as the foundation for further development.

The **Design Report** includes system architecture, DFDs, ER diagrams, and descriptions of key modules such as user management, blog management, media handling, and content display. It provides a clear understanding of how different components interact and how data flows within the system.

The **Implementation Report** explains the tools and technologies used, such as frontend and backend frameworks and database systems. It describes module-wise development, integration between components, and the overall workflow of the system.

The **Testing Report** contains details of unit testing, integration testing, and system testing performed on the website. It includes test cases, expected and actual outputs, and validation results to ensure system reliability and performance.

Finally, the **Project Completion Report** summarizes the achievements, system performance, limitations, and possible future enhancements. It provides an overall evaluation of how successfully the Band Blogging Website meets its objectives.

9. FUTURE SCOPE

i) Support for Multiple Bands and Categories:

The Band Blogging Website has strong potential for future expansion. Currently, the system may focus on a single band or limited content, but it can be extended to support multiple bands and categories. This would allow different bands to create and manage their own profiles, blogs, tours, and discography. Adding categories such as genres, events, or news sections would improve content organization and make the platform more useful for a wider audience.

ii) Integration with Social Media Platforms:

Another important future enhancement is integration with social media platforms. By connecting the website with platforms like Instagram, Facebook, or Twitter, bands can automatically share blog updates, tour announcements, and media content. This would increase visibility, improve audience engagement, and reduce the effort required to manage multiple platforms separately.

iii) Advanced User Interaction and Personalization:

A further improvement involves enhancing user interaction and personalization features. The system can include user accounts with personalized dashboards, allowing users to follow their favorite bands, receive notifications, and get customized content recommendations. Features such as likes, sharing options, and real-time comments can further improve engagement and user experience.

Overall:

These future enhancements would make the Band Blogging Website more dynamic, scalable, and interactive. They would expand its usability, improve user engagement, and transform it into a more comprehensive platform for both bands and their audience.

10. APPENDICES

10.1 CODING

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Hydes Garage</title>

  <link
href="https://fonts.googleapis.com/css2?family=Permanent+Marker&family=Comic+Neue:wght@700&display=sw
ap" rel="stylesheet">

  <style>
    * { box-sizing: border-box; }

    body {
      margin: 0;
      font-family: 'Comic Neue', cursive;
      background: #0f0f14;
      color: #f5f5f5;
      background-image: repeating-linear-gradient(
        -45deg,
        #14141f 0px,
        #14141f 12px,
        #0f0f14 12px,
        #0f0f14 24px
      );
    }

    /* HEADER */
    header {
      background: linear-gradient(135deg, #baffc9, #fef6a7);
      color: #000;
      padding: 3rem 1rem 2.5rem;
      text-align: center;
```

```

font-family: 'Permanent Marker', cursive;
transform: rotate(-1.5deg);
border-bottom: 6px dashed #000;
}

```

```

/* POLAROID BAND PHOTO */

```

```

.polaroid {
width: 180px;
background: #fff;
margin: 0 auto 1.5rem;
padding: 0.8rem 0.8rem 1.6rem;
border: 3px solid #000;
box-shadow: 8px 8px 0 #000;
transform: rotate(-4deg);
}

```

```

.polaroid-photo {
width: 100%;
height: 140px;
border: 3px solid #000;
border-radius: 50%;
overflow: hidden;
}

```

```

.polaroid-photo img {
width: 100%;
height: 100%;
object-fit: cover;
}

```

```

.polaroid-caption {
margin-top: 0.8rem;
font-size: 0.75rem;
letter-spacing: 1px;
}

```

```

header h1 {

```

```

    font-size: 3.8rem;
    margin: 0;
}

header p {
    margin-top: 0.5rem;
    font-size: 1.1rem;
}

/* NAV */
nav {
    background: #ff6ec7;
    display: flex;
    justify-content: center;
    gap: 1.5rem;
    padding: 1rem;
    border-bottom: 4px solid #000;
}

nav a {
    text-decoration: none;
    color: #000;
    background: #fff;
    padding: 0.5rem 0.9rem;
    border: 2px solid #000;
    font-weight: bold;
    transform: rotate(2deg);
    box-shadow: 3px 3px 0 #000;
}

nav a:hover { background: #baffc9; }

/* MAIN */
main {
    max-width: 1050px;
    margin: auto;
    padding: 2rem;

```

```

display: grid;
grid-template-columns: 3fr 1fr;
gap: 2rem;
}

.post {
background: #181825;
padding: 1.6rem;
margin-bottom: 2.2rem;
border: 4px solid #fff;
box-shadow: 10px 10px 0 #ff6ec7;
transform: rotate(-0.8deg);
}

.post:nth-child(even) {
transform: rotate(0.6deg);
box-shadow: 10px 10px 0 #baffc9;
}

.post h2 {
font-family: 'Permanent Marker', cursive;
color: #fef6a7;
margin-top: 0;
}

.date {
font-size: 0.85rem;
color: #ffb7e5;
margin-bottom: 1rem;
}

.post-content {
display: flex;
gap: 1.5rem;
flex-wrap: wrap;
}

```



```

.sticker {
  display: inline-block;
  background: #ff6ec7;
  color: #000;
  padding: 0.3rem 0.6rem;
  font-size: 0.75rem;
  border: 2px solid #000;
  transform: rotate(-6deg);
  margin-right: 0.5rem;
}

/* ALBUM / SINGLE THUMBNAILS */
.thumb {
  width: 160px;
  height: 160px;
  border: 4px solid #000;
  box-shadow: 6px 6px 0 #000;
  transform: rotate(-3deg);
  flex-shrink: 0;
  overflow: hidden;
}

.thumb.single { transform: rotate(2deg); }

.thumb img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

/* ASIDE */
aside {
  background: #fef6a7;
  color: #000;
  padding: 1.5rem;
  border: 4px solid #000;
  box-shadow: 7px 7px 0 #000;
}

```

```
    transform: rotate(1.2deg);  
}
```

```
aside h3 {  
    font-family: 'Permanent Marker', cursive;  
    margin-top: 0;  
}
```

```
.listen {  
    margin-top: 1.5rem;  
    padding-top: 1rem;  
    border-top: 3px dashed #000;  
}
```

```
.listen a {  
    display: block;  
    margin-top: 0.6rem;  
    text-decoration: none;  
    background: #fff;  
    color: #000;  
    padding: 0.5rem;  
    border: 2px solid #000;  
    text-align: center;  
    font-weight: bold;  
    box-shadow: 3px 3px 0 #000;  
}
```

```
.listen a:hover { background: #baffc9; }
```

```
footer {  
    text-align: center;  
    padding: 1.5rem;  
    background: #000;  
    border-top: 4px dashed #fef6a7;  
    font-size: 0.85rem;  
}
```

```

    @media (max-width: 768px) {
      main { grid-template-columns: 1fr; }
      .post-content { justify-content: center; }
    }
  </style>
</head>

<body>

<header>
  <div class="polaroid">
    <div class="polaroid-photo">
      <!-- BAND PHOTO -->
      
    </div>
    <div class="polaroid-caption">SERKAWN, MZ • EST. 2020</div>
  </div>

  <h1>HYDES GARAGE</h1>
  <p> ⚡ indie • sensation ⚡ </p>
</header>

<nav>
  <a href="#">Blog</a>
  <a href="#">Albums</a>
  <a href="#">Tour Chaos</a>
  <a href="#">Fan Rants</a>
</nav>

<main>
  <section>

    <!-- ALBUM -->
    <article class="post">
      <h2>Airborne Deceptions</h2>
      <div class="date">Aug 14, 2025</div>

```

```

<div class="post-content">
  <div class="thumb">
    
  </div>

  <p>
    <span class="sticker">ALBUM</span>
    With the release of their debut album, Hyde's Garage, a promising independent
    band from Serkawn, Mizoram caught the attention of music lovers nationwide.
    Refreshing beats, playful structures, and a fearless DIY spirit define their sound.
  </p>
</div>
</article>

```

```

<!-- SINGLE -->
<article class="post">
  <h2>Ah! Ka thian</h2>
  <div class="date">Sep 25, 2020</div>

  <div class="post-content">
    <div class="thumb single">
      
    </div>

```

```

    <p>
      <span class="sticker">SINGLE</span>
      Hyde's Garage's first single raw, heartfelt, and unintentionally iconic.
      What started as a friends-only project quietly sparked a creative journey
      full of charm, chaos, and late-night bedroom recordings.
    </p>
  </div>
</article>

```

```

</section>

```

```

<aside>
  <h3>△ NOTES</h3>

```

```

<ul>
  <li>🎧 Favorite Album: Airborne Deceptions</li>
  <li>🔥 Alignment: Chaotic Good</li>
  <li>🎵 Vibes: whimsical, lo-fi, indie dreams</li>
  <li>✍️ DIY > Perfect</li>
</ul>

<div class="listen">
  <h3>🎧 LISTEN HERE</h3>
  <a href="https://www.youtube.com/@hydesgarage" target="_blank">▶ YouTube Channel</a>
  <a href="https://open.spotify.com/artist/44qrg8E3Ett3lxLqNzkazN" target="_blank">🎧 Spotify</a>
</div>
</aside>
</main>

<footer>
  <p>Unofficial Fanpage • Indie Forever • Made with love & noise</p>
</footer>

</body>
</html>

```

About.js:

```
import { Router, type IRouter } from "express";
import { db, aboutTable } from "@workspace/db";
import { eq } from "drizzle-orm";
import { UpdateAboutBody } from "@workspace/api-zod";
import { requireAuth } from "../middlewares/requireAuth";

const router: IRouter = Router();

function serializeAbout(row: typeof aboutTable.$inferSelect) {
  return {
    id: row.id,
    bandName: row.bandName,
    tagline: row.tagline,
    story: row.story,
    hometown: row.hometown,
    foundedYear: row.foundedYear,
    photoUrl: row.photoUrl,
    youtubeUrl: row.youtubeUrl,
    spotifyUrl: row.spotifyUrl,
    instagramUrl: row.instagramUrl,
    updatedAt: row.updatedAt.toISOString(),
  };
}

const DEFAULT_ABOUT = {
  bandName: "Hydes Garage",
  tagline: "indie sensation from the garage",
  story:
    "We are Hydes Garage, an independent band from Serkawn, Mizoram. We make scrappy, lo-fi, heart-on-sleeve music in bedrooms and back rooms.",
  hometown: "Serkawn, Mizoram",
  foundedYear: 2020,
  photoUrl: null as string | null,
  youtubeUrl: null as string | null,
  spotifyUrl: null as string | null,
  instagramUrl: null as string | null,
};
```

```

async function ensureAbout() {
  const rows = await db.select().from(aboutTable).limit(1);
  if (rows[0]) return rows[0];
  const [row] = await db.insert(aboutTable).values(DEFAULT_ABOUT).returning();
  return row;
}

router.get("/about", async (_req, res) => {
  const row = await ensureAbout();
  res.json(serializeAbout(row));
});

router.put("/about", requireAuth, async (req, res) => {
  const parsed = UpdateAboutBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid about payload" });
    return;
  }
  const existing = await ensureAbout();
  const [row] = await db
    .update(aboutTable)
    .set({
      bandName: parsed.data.bandName,
      tagline: parsed.data.tagline,
      story: parsed.data.story,
      hometown: parsed.data.hometown,
      foundedYear: parsed.data.foundedYear,
      photoUrl: parsed.data.photoUrl ?? null,
      youtubeUrl: parsed.data.youtubeUrl ?? null,
      spotifyUrl: parsed.data.spotifyUrl ?? null,
      instagramUrl: parsed.data.instagramUrl ?? null,
      updatedAt: new Date(),
    })
    .where(eq(aboutTable.id, existing.id))
    .returning();
  res.json(serializeAbout(row));
});

export default router;

```

albums.js:

```
import { Router, type IRouter } from "express";
import { db, albumsTable } from "@workspace/db";
import { desc, eq } from "drizzle-orm";
import {
  CreateAlbumBody,
  GetAlbumParams,
  UpdateAlbumBody,
  UpdateAlbumParams,
  DeleteAlbumParams,
} from "@workspace/api-zod";
import { requireAuth } from "../middlewares/requireAuth";

const router: IRouter = Router();

function serializeAlbum(row: typeof albumsTable.$inferSelect) {
  return {
    id: row.id,
    title: row.title,
    releaseDate: row.releaseDate.toISOString(),
    description: row.description,
    coverUrl: row.coverUrl,
    listenUrl: row.listenUrl,
    createdAt: row.createdAt.toISOString(),
  };
}

router.get("/albums", async (_req, res) => {
  const rows = await db
    .select()
    .from(albumsTable)
    .orderBy(desc(albumsTable.releaseDate));
  res.json(rows.map(serializeAlbum));
});

router.get("/albums/:id", async (req, res) => {
  const params = GetAlbumParams.safeParse({ id: Number(req.params.id) });
```



```

if (!params.success) {
  res.status(400).json({ error: "Invalid id" });
  return;
}
const [row] = await db
  .select()
  .from(albumsTable)
  .where(eq(albumsTable.id, params.data.id));
if (!row) {
  res.status(404).json({ error: "Album not found" });
  return;
}
res.json(serializeAlbum(row));
});

router.post("/albums", requireAuth, async (req, res) => {
  const parsed = CreateAlbumBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid album payload" });
    return;
  }
  const [row] = await db
    .insert(albumsTable)
    .values({
      title: parsed.data.title,
      releaseDate: new Date(parsed.data.releaseDate),
      description: parsed.data.description,
      coverUrl: parsed.data.coverUrl ?? null,
      listenUrl: parsed.data.listenUrl ?? null,
    })
    .returning();
  res.status(201).json(serializeAlbum(row));
});

router.patch("/albums/:id", requireAuth, async (req, res) => {
  const params = UpdateAlbumParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {

```

```

    res.status(400).json({ error: "Invalid id" });
    return;
  }
  const parsed = UpdateAlbumBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid album payload" });
    return;
  }
  const [row] = await db
    .update(albumsTable)
    .set({
      title: parsed.data.title,
      releaseDate: new Date(parsed.data.releaseDate),
      description: parsed.data.description,
      coverUrl: parsed.data.coverUrl ?? null,
      listenUrl: parsed.data.listenUrl ?? null,
    })
    .where(eq(albumsTable.id, params.data.id))
    .returning();
  if (!row) {
    res.status(404).json({ error: "Album not found" });
    return;
  }
  res.json(serializeAlbum(row));
});

router.delete("/albums/:id", requireAuth, async (req, res) => {
  const params = DeleteAlbumParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {
    res.status(400).json({ error: "Invalid id" });
    return;
  }
  await db.delete(albumsTable).where(eq(albumsTable.id, params.data.id));
  res.status(204).end();
});

export default router;

```

shows.js:

```
import { Router, type IRouter } from "express";
import { db, showsTable } from "@workspace/db";
import { asc, desc, eq, gte } from "drizzle-orm";
import {
  CreateShowBody,
  GetShowParams,
  UpdateShowBody,
  UpdateShowParams,
  DeleteShowParams,
} from "@workspace/api-zod";
import { requireAuth } from "../middlewares/requireAuth";
const router: IRouter = Router();
function serializeShow(row: typeof showsTable.$inferSelect) {
  return {
    id: row.id,
    title: row.title,
    venue: row.venue,
    city: row.city,
    date: row.date.toISOString(),
    ticketUrl: row.ticketUrl,
    notes: row.notes,
    createdAt: row.createdAt.toISOString(),
  };
}
router.get("/shows", async (_req, res) => {
  const rows = await db.select().from(showsTable).orderBy(desc(showsTable.date));
  res.json(rows.map(serializeShow));
});
router.get("/shows/upcoming", async (_req, res) => {
  const now = new Date();
  const rows = await db
    .select()
    .from(showsTable)
    .where(gte(showsTable.date, now))
    .orderBy(asc(showsTable.date));
  res.json(rows.map(serializeShow));
});
```

```

});
router.get("/shows/:id", async (req, res) => {
  const params = GetShowParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {
    res.status(400).json({ error: "Invalid id" });
    return;
  }
  const [row] = await db
    .select()
    .from(showsTable)
    .where(eq(showsTable.id, params.data.id));
  if (!row) {
    res.status(404).json({ error: "Show not found" });
    return;
  }
  res.json(serializeShow(row));
});
router.post("/shows", requireAuth, async (req, res) => {
  const parsed = CreateShowBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid show payload" });
    return;
  }
  const [row] = await db
    .insert(showsTable)
    .values({
      title: parsed.data.title,
      venue: parsed.data.venue,
      city: parsed.data.city,
      date: new Date(parsed.data.date),
      ticketUrl: parsed.data.ticketUrl ?? null,
      notes: parsed.data.notes ?? null,
    })
    .returning();
  res.status(201).json(serializeShow(row));
});

```

```

router.patch("/shows/:id", requireAuth, async (req, res) => {
  const params = UpdateShowParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {
    res.status(400).json({ error: "Invalid id" });
    return;
  }
  const parsed = UpdateShowBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid show payload" });
    return;
  }
  const [row] = await db
    .update(showsTable)
    .set({
      title: parsed.data.title,
      venue: parsed.data.venue,
      city: parsed.data.city,
      date: new Date(parsed.data.date),
      ticketUrl: parsed.data.ticketUrl ?? null,
      notes: parsed.data.notes ?? null,
    })
    .where(eq(showsTable.id, params.data.id))
    .returning();
  if (!row) {
    res.status(404).json({ error: "Show not found" });
    return;
  }
  res.json(serializeShow(row));
});

router.delete("/shows/:id", requireAuth, async (req, res) => {
  const params = DeleteShowParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {
    res.status(400).json({ error: "Invalid id" });
    return;
  }
  await db.delete(showsTable).where(eq(showsTable.id, params.data.id));
  res.status(204).end(); }); export default router;

```

singles.js:

```
import { Router, type IRouter } from "express";
import { db, singlesTable } from "@workspace/db";
import { desc, eq } from "drizzle-orm";
import {
  CreateSingleBody,
  GetSingleParams,
  UpdateSingleBody,
  UpdateSingleParams,
  DeleteSingleParams,
} from "@workspace/api-zod";
import { requireAuth } from "../middlewares/requireAuth";

const router: IRouter = Router();

function serializeSingle(row: typeof singlesTable.$inferSelect) {
  return {
    id: row.id,
    title: row.title,
    releaseDate: row.releaseDate.toISOString(),
    description: row.description,
    coverUrl: row.coverUrl,
    listenUrl: row.listenUrl,
    createdAt: row.createdAt.toISOString(),
  };
}

router.get("/singles", async (_req, res) => {
  const rows = await db
    .select()
    .from(singlesTable)
    .orderBy(desc(singlesTable.releaseDate));
  res.json(rows.map(serializeSingle));
});

router.get("/singles/:id", async (req, res) => {
  const params = GetSingleParams.safeParse({ id: Number(req.params.id) });
```

```

if (!params.success) {
  res.status(400).json({ error: "Invalid id" });
  return;
}
const [row] = await db
  .select()
  .from(singlesTable)
  .where(eq(singlesTable.id, params.data.id));
if (!row) {
  res.status(404).json({ error: "Single not found" });
  return;
}
res.json(serializeSingle(row));
});

router.post("/singles", requireAuth, async (req, res) => {
  const parsed = CreateSingleBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid single payload" });
    return;
  }
  const [row] = await db
    .insert(singlesTable)
    .values({
      title: parsed.data.title,
      releaseDate: new Date(parsed.data.releaseDate),
      description: parsed.data.description,
      coverUrl: parsed.data.coverUrl ?? null,
      listenUrl: parsed.data.listenUrl ?? null,
    })
    .returning();
  res.status(201).json(serializeSingle(row));
});

router.patch("/singles/:id", requireAuth, async (req, res) => {
  const params = UpdateSingleParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {

```

```

    res.status(400).json({ error: "Invalid id" });
    return;
  }
  const parsed = UpdateSingleBody.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid single payload" });
    return;
  }
  const [row] = await db
    .update(singlesTable)
    .set({
      title: parsed.data.title,
      releaseDate: new Date(parsed.data.releaseDate),
      description: parsed.data.description,
      coverUrl: parsed.data.coverUrl ?? null,
      listenUrl: parsed.data.listenUrl ?? null,
    })
    .where(eq(singlesTable.id, params.data.id))
    .returning();
  if (!row) {
    res.status(404).json({ error: "Single not found" });
    return;
  }
  res.json(serializeSingle(row));
});

router.delete("/singles/:id", requireAuth, async (req, res) => {
  const params = DeleteSingleParams.safeParse({ id: Number(req.params.id) });
  if (!params.success) {
    res.status(400).json({ error: "Invalid id" });
    return;
  }
  await db.delete(singlesTable).where(eq(singlesTable.id, params.data.id));
  res.status(204).end();
});

export default router;

```


Admin Panel:

Login Page:

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { useLogin, getGetMeQueryKey } from "@workspace/api-client-react";
import { useQueryClient } from "@tanstack/react-query";
import { useLocation } from "wouter";
import { toast } from "sonner";
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
import { Input } from "@components/ui/input";
import { Link } from "wouter";

const loginSchema = z.object({
  username: z.string().min(1, "Username is required"),
  password: z.string().min(1, "Password is required"),
});

type LoginForm = z.infer<typeof loginSchema>;

export default function Login() {
  const [, setLocation] = useLocation();
  const queryClient = useQueryClient();
  const login = useLogin();
  const form = useForm<LoginForm>({
    resolver: zodResolver(loginSchema),
    defaultValues: {
      username: "",
      password: "",
    },
  });

  const onSubmit = (data: LoginForm) => {
    login.mutate({ data }, {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: getGetMeQueryKey() });
        toast.success("Welcome to the backroom.");
        setLocation("/admin");
      },
      onError: (error) => {
        toast.error(error.message || "Failed to login. Check your credentials.");
      }
    });
  }
}
```

```

    }
  });
};
return (
  <div className="min-h-screen bg-black flex flex-col items-center justify-center p-4 relative overflow-hidden">
    { /* Background decoration */ }
    <div className="absolute inset-0 bg-[url('https://www.transparenttextures.com/patterns/stardust.png')] opacity-20 pointer-events-none"></div>
    <Link href="/" className="absolute top-8 left-8 text-white uppercase font-bold hover:text-primary tracking-widest text-sm z-10">
      ← Back to site
    </Link>
    <div className="bg-card w-full max-w-md border-4 border-white p-8 shadow-[12px_12px_0px_0px_#ff6ec7] rotate-1 relative z-10">
      <div className="absolute -top-4 -left-4 w-12 h-12 bg-accent border-2 border-black rotate-12 flex items-center justify-center font-display text-2xl text-black shadow-[2px_2px_0_0_#000]">
        !
      </div>
      <h1 className="font-display text-4xl text-center mb-2 uppercase text-white">Authorized<br/>Personnel<br/>Only</h1>
      <p className="text-center text-muted-foreground mb-8 text-sm uppercase tracking-widest border-b-2 border-dashed border-gray-600 pb-4">Hydes Garage Admin</p>
      <Form {...form}>
        <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-6">
          <FormField
            control={form.control}
            name="username"
            render={({ field }) => (
              <FormItem>
                <FormLabel className="uppercase tracking-widest text-white text-xs">Username</FormLabel>
                <FormControl>
                  <Input {...field} className="bg-black border-2 border-white text-white rounded-none h-12 focus-visible:ring-primary focus-visible:ring-offset-0 focus-visible:border-primary" />
                </FormControl>
                <FormMessage className="text-primary" />
              </FormItem>
            )}
          </form>
        </Form>
      </div>
    </div>
  </div>
);

```

```

/>
<FormField
  control={form.control}
  name="password"
  render={({ field }) => (
    <FormItem>
      <FormLabel className="uppercase tracking-widest text-white text-xs">Password</FormLabel>
      <FormControl>
        <Input type="password" {...field} className="bg-black border-2 border-white text-white rounded-
none h-12 focus-visible:ring-primary focus-visible:ring-offset-0 focus-visible:border-primary" />
      </FormControl>
      <FormMessage className="text-primary" />
    </FormItem>
  )}
/>
<button
  type="submit"
  disabled={login.isPending}
  className="w-full bg-white text-black font-display text-xl py-3 border-2 border-black shadow-
[4px_4px_0_0_#baffc9] hover:translate-y-[2px] hover:shadow-[2px_2px_0_0_#baffc9] active:translate-y-[4px]
active:shadow-none transition-all uppercase mt-4 disabled:opacity-50"
  >
    {login.isPending ? "Unlocking..." : "Enter"}
  </button>
</form>
</Form>
</div>
</div>
);
}

```

Admin Dashboard:

```
import { AdminLayout } from "@components/layout/AdminLayout";
import { useGetSummary } from "@workspace/api-client-react";
import { Link } from "wouter";
export default function Dashboard() {
  const { data: summary, isLoading } = useGetSummary();
  return (
    <AdminLayout>
      <h2 className="font-display text-4xl mb-8 uppercase text-primary drop-shadow-md">Command Center</h2>
      {isLoading ? (
        <div className="flex justify-center py-12">
          <div className="animate-spin h-12 w-12 border-8 border-accent border-t-transparent rounded-full"></div>
        </div>
      ) : summary ? (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-12">
          <StatCard title="Total Shows" value={summary.showCount} color="bg-primary" route="/admin/shows" />
          <StatCard title="Upcoming Shows" value={summary.upcomingShowCount} color="bg-accent"
route="/admin/shows" />
          <StatCard title="Albums" value={summary.albumCount} color="bg-secondary" route="/admin/albums" />
          <StatCard title="Singles" value={summary.singleCount} color="bg-white" route="/admin/singles" />
        </div>
      ) : null}
      {summary?.nextShow && (
        <div className="bg-card border-4 border-white p-6 shadow-[8px_8px_0px_0px_#ff6ec7] rotate-1 max-w-
2xl">
          <span className="sticker-badge mb-2">NEXT SHOW</span>
          <h3 className="font-display text-2xl uppercase mt-2">{summary.nextShow.city}</h3>
          <p className="text-lg text-muted-foreground">{summary.nextShow.venue}</p>
          <p className="font-bold text-primary tracking-widest uppercase mt-2">
            {new Date(summary.nextShow.date).toLocaleString(undefined, { weekday: 'short', month: 'short', day:
'numeric', hour: 'numeric', minute: '2-digit' })}
          </p>
        </div>
      )}
    </AdminLayout>
  );
}
```

```

function StatCard({ title, value, color, route }: { title: string, value: number, color: string, route: string }) {
  return (
    <Link href={route} className={` ${color} text-black p-6 border-4 border-black shadow-
[6px_6px_0px_0px_#000] hover:-translate-y-1 hover:shadow-[8px_8px_0px_0px_#000] transition-all flex flex-col
justify-between block`} >
      <h3 className="font-bold uppercase tracking-widest text-sm mb-4">{title}</h3>
      <div className="font-display text-5xl">{value}</div>
    </Link>
  );
}

```

Manage Shows Page:

```
import { useState } from "react";
import { AdminLayout } from "@components/layout/AdminLayout";
import { useListShows, useCreateShow, useUpdateShow, useDeleteShow, getListShowsQueryKey,
getGetSummaryQueryKey } from "@workspace/api-client-react";
import { useQueryClient } from "@tanstack/react-query";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { toast } from "sonner";
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription,
AlertDialogFooter, AlertDialogHeader, AlertDialogTitle, AlertDialogTrigger } from "@components/ui/alert-
dialog";
const showSchema = z.object({
  title: z.string().min(1, "Required"),
  venue: z.string().min(1, "Required"),
  city: z.string().min(1, "Required"),
  date: z.string().min(1, "Required"),
  ticketUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  notes: z.string().optional().or(z.literal("")),
});
type ShowForm = z.infer<typeof showSchema>;
export default function AdminShows() {
  const queryClient = useQueryClient();
  const { data: shows = [], isLoading } = useListShows();
  const createShow = useCreateShow();
  const updateShow = useUpdateShow();
  const deleteShow = useDeleteShow();
  const [editingId, setEditingId] = useState<number | null>(null);
  const [isFormOpen, setIsFormOpen] = useState(false);

  const form = useForm<ShowForm>({
    resolver: zodResolver(showSchema),
    defaultValues: { title: "", venue: "", city: "", date: "", ticketUrl: "", notes: "" }
  });
```

```

});
const openCreate = () => {
  setEditingId(null);
  form.reset({ title: "", venue: "", city: "", date: "", ticketUrl: "", notes: "" });
  setIsFormOpen(true);
};
const openEdit = (show: any) => {
  setEditingId(show.id);
  // Format date for datetime-local input
  const dateObj = new Date(show.date);
  const tzoffset = dateObj.getTimezoneOffset() * 60000; // offset in milliseconds
  const localISOTime = (new Date(dateObj.getTime() - tzoffset)).toISOString().slice(0,16);
  form.reset({
    title: show.title,
    venue: show.venue,
    city: show.city,
    date: localISOTime,
    ticketUrl: show.ticketUrl || "",
    notes: show.notes || ""
  });
  setIsFormOpen(true);
};
const onSubmit = (data: ShowForm) => {
  const payload = {
    ...data,
    date: new Date(data.date).toISOString(), // ensure ISO format
    ticketUrl: data.ticketUrl || null,
    notes: data.notes || null,
  };
  if (editingId) {
    updateShow.mutate({ id: editingId, data: payload }, {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: getListShowsQueryKey() });
        queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
        toast.success("Show updated!");
        setIsFormOpen(false);
      }
    });
  }
};

```

```

    },
    onError: (err) => toast.error(err.message)
  });
} else {
  createShow.mutate({ data: payload }, {
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: getListShowsQueryKey() });
      queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
      toast.success("Show created!");
      setIsFormOpen(false);
    },
    onError: (err) => toast.error(err.message)
  });
}
};

const handleDelete = (id: number) => {
  deleteShow.mutate({ id }, {
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: getListShowsQueryKey() });
      queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
      toast.success("Show deleted.");
    },
    onError: (err) => toast.error(err.message)
  });
};

return (
  <AdminLayout>
    <div className="flex justify-between items-center mb-8">
      <h2 className="font-display text-4xl text-primary uppercase">Manage Shows</h2>
      {!isFormOpen && (
        <button onClick={openCreate} className="bg-white text-black border-2 border-black font-bold uppercase
px-4 py-2 shadow-[4px_4px_0_0_#baffc9] hover:translate-y-[2px] hover:shadow-[2px_2px_0_0_#baffc9]">
          + Add Show
        </button>
      )}
    </div>
    {isFormOpen ? (

```



```

<div className="bg-card border-4 border-white p-6 shadow-[8px_8px_0_0_#ff6ec7] mb-8 rotate-[0.5deg]">
  <h3 className="font-display text-2xl uppercase mb-4">{editingId ? 'Edit Show' : 'New Show'}</h3>
  <Form {...form}>
    <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        <FormField control={form.control} name="title" render={({ field }) => (
          <FormItem><FormLabel>Tour/Show Title</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
        )} />
        <FormField control={form.control} name="date" render={({ field }) => (
          <FormItem><FormLabel>Date & Time</FormLabel><FormControl><Input type="datetime-local"
{...field} className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
        )} />
        <FormField control={form.control} name="venue" render={({ field }) => (
          <FormItem><FormLabel>Venue</FormLabel><FormControl><Input {...field} className="bg-black
border-white" /></FormControl><FormMessage /></FormItem>
        )} />
        <FormField control={form.control} name="city" render={({ field }) => (
          <FormItem><FormLabel>City</FormLabel><FormControl><Input {...field} className="bg-black
border-white" /></FormControl><FormMessage /></FormItem>
        )} />
        <FormField control={form.control} name="ticketUrl" render={({ field }) => (
          <FormItem><FormLabel>Ticket URL (optional)</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
        )} />
      </div>
      <FormField control={form.control} name="notes" render={({ field }) => (
        <FormItem><FormLabel>Notes (optional)</FormLabel><FormControl><Textarea {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
      )} />
      <div className="flex gap-4 pt-4">
        <button type="submit" disabled={createShow.isPending || updateShow.isPending} className="bg-
primary text-black font-bold uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-
translate-y-1">
          Save Show
        </button>

```

```

        <button type="button" onClick={() => setIsFormOpen(false)} className="bg-white text-black font-bold
uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-translate-y-1">
            Cancel
        </button>
    </div>
</form>
</Form>
</div>
): (
    <div className="space-y-4">
        {isLoading ? (
            <div className="flex justify-center p-8"><div className="animate-spin h-8 w-8 border-4 border-primary
border-t-transparent rounded-full"></div></div>
        ) : shows.length > 0 ? (
            shows.sort((a,b) => new Date(b.date).getTime() - new Date(a.date).getTime()).map(show => (
                <div key={show.id} className="flex flex-col sm:flex-row sm:items-center justify-between bg-black
border-2 border-white p-4 hover:border-primary transition-colors">
                    <div>
                        <div className="text-primary font-bold text-sm tracking-widest">{new
Date(show.date).toLocaleString()}</div>
                        <div className="text-xl font-display uppercase">{show.title}</div>
                        <div className="text-muted-foreground">{show.venue} • {show.city}</div>
                    </div>
                    <div className="flex gap-2 mt-4 sm:mt-0">
                        <button onClick={() => openEdit(show)} className="bg-white text-black px-3 py-1 border-2 border-
black font-bold text-sm uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Edit</button>
                        <AlertDialog>
                            <AlertDialogTrigger asChild>
                                <button className="bg-destructive text-white px-3 py-1 border-2 border-black font-bold text-sm
uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Delete</button>
                            </AlertDialogTrigger>
                            <AlertDialogContent className="bg-card border-4 border-white rounded-none">
                                <AlertDialogHeader>
                                    <AlertDialogTitle className="font-display uppercase text-2xl text-destructive">Delete
Show?</AlertDialogTitle>
                                    <AlertDialogDescription className="text-white">This cannot be undone. Are you
sure?</AlertDialogDescription>

```


Manage Albums Page:

```
import { useState } from "react";
import { AdminLayout } from "@components/layout/AdminLayout";
import { useListAlbums, useCreateAlbum, useUpdateAlbum, useDeleteAlbum, getListAlbumsQueryKey,
  getGetSummaryQueryKey } from "@workspace/api-client-react";
import { useQueryClient } from "@tanstack/react-query";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { toast } from "sonner";
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle, AlertDialogTrigger } from "@components/ui/alert-
  dialog";
const albumSchema = z.object({
  title: z.string().min(1, "Required"),
  releaseDate: z.string().min(1, "Required"),
  description: z.string().min(1, "Required"),
  coverUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  listenUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
});
type AlbumForm = z.infer<typeof albumSchema>;
export default function AdminAlbums() {
  const queryClient = useQueryClient();
  const { data: albums = [], isLoading } = useListAlbums();
  const createAlbum = useCreateAlbum();
  const updateAlbum = useUpdateAlbum();
  const deleteAlbum = useDeleteAlbum();
  const [editingId, setEditingId] = useState<number | null>(null);
  const [isFormOpen, setIsFormOpen] = useState(false);
  const form = useForm<AlbumForm>({
    resolver: zodResolver(albumSchema),
    defaultValues: { title: "", releaseDate: "", description: "", coverUrl: "", listenUrl: "" }
  });
  const openCreate = () => {
```

```

setEditingId(null);
form.reset({ title: "", releaseDate: "", description: "", coverUrl: "", listenUrl: "" });
setIsFormOpen(true);
};

const openEdit = (album: any) => {
  setEditingId(album.id);
  const dateObj = new Date(album.releaseDate);
  const tzoffset = dateObj.getTimezoneOffset() * 60000;
  const localISOTime = (new Date(dateObj.getTime() - tzoffset)).toISOString().slice(0,16);
  form.reset({
    title: album.title,
    releaseDate: localISOTime,
    description: album.description,
    coverUrl: album.coverUrl || "",
    listenUrl: album.listenUrl || ""
  });
  setIsFormOpen(true);
};

const onSubmit = (data: AlbumForm) => {
  const payload = {
    ...data,
    releaseDate: new Date(data.releaseDate).toISOString(),
    coverUrl: data.coverUrl || null,
    listenUrl: data.listenUrl || null,
  };
  if (editingId) {
    updateAlbum.mutate({ id: editingId, data: payload }, {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: getListAlbumsQueryKey() });
        queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
        toast.success("Album updated!");
        setIsFormOpen(false);
      },
      onError: (err) => toast.error(err.message)
    });
  } else {
    createAlbum.mutate({ data: payload }, {

```

```

onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: getListAlbumsQueryKey() });
  queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
  toast.success("Album created!");
  setIsFormOpen(false);
},
onError: (err) => toast.error(err.message)
});
}
};

const handleDelete = (id: number) => {
  deleteAlbum.mutate({ id }, {
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: getListAlbumsQueryKey() });
      queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
      toast.success("Album deleted.");
    },
    onError: (err) => toast.error(err.message)
  });
};

return (
  <AdminLayout>
    <div className="flex justify-between items-center mb-8">
      <h2 className="font-display text-4xl text-secondary uppercase">Manage Albums</h2>
      {!isFormOpen && (
        <button onClick={openCreate} className="bg-white text-black border-2 border-black font-bold uppercase
px-4 py-2 shadow-[4px_4px_0_0_#ff6ec7] hover:translate-y-[-2px] hover:shadow-[2px_2px_0_0_#ff6ec7]">
          + Add Album
        </button>
      )}
    </div>
    {isFormOpen ? (
      <div className="bg-card border-4 border-white p-6 shadow-[8px_8px_0_0_#baffc9] mb-8 -rotate-[0.5deg]">
        <h3 className="font-display text-2xl uppercase mb-4">{editingId ? 'Edit Album' : 'New Album'}</h3>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

```

```

    <FormField control={form.control} name="title" render={({ field }) => (
      <FormItem><FormLabel>Album Title</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="releaseDate" render={({ field }) => (
      <FormItem><FormLabel>Release Date</FormLabel><FormControl><Input type="datetime-local"
{...field} className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="coverUrl" render={({ field }) => (
      <FormItem><FormLabel>Cover Art URL</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="listenUrl" render={({ field }) => (
      <FormItem><FormLabel>Streaming URL</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
  </div>
  <FormField control={form.control} name="description" render={({ field }) => (
    <FormItem><FormLabel>Description</FormLabel><FormControl><Textarea rows={4} {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
  )} />
  <div className="flex gap-4 pt-4">
    <button type="submit" disabled={createAlbum.isPending || updateAlbum.isPending} className="bg-
secondary text-black font-bold uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-
translate-y-1">
      Save Album
    </button>
    <button type="button" onClick={() => setIsFormOpen(false)} className="bg-white text-black font-bold
uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-translate-y-1">
      Cancel
    </button>
  </div>
</form>
</Form>
</div>
):(
  <div className="space-y-4">

```

```

{isLoading ? (
  <div className="flex justify-center p-8"><div className="animate-spin h-8 w-8 border-4 border-
secondary border-t-transparent rounded-full"></div></div>
) : albums.length > 0 ? (
  albums.sort((a,b) => new Date(b.releaseDate).getTime() - new Date(a.releaseDate).getTime()).map(album
=> (
    <div key={album.id} className="flex flex-col sm:flex-row sm:items-center justify-between bg-black
border-2 border-white p-4 hover:border-secondary transition-colors">
      <div className="flex gap-4 items-center">
        {album.coverUrl && <img src={album.coverUrl} alt={album.title} className="w-16 h-16 object-cover
border-2 border-white" />}
      <div>
        <div className="text-secondary font-bold text-sm tracking-widest">{new
Date(album.releaseDate).toLocaleDateString()}</div>
        <div className="text-xl font-display uppercase">{album.title}</div>
      </div>
    </div>
    <div className="flex gap-2 mt-4 sm:mt-0">
      <button onClick={() => openEdit(album)} className="bg-white text-black px-3 py-1 border-2 border-
black font-bold text-sm uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Edit</button>
      <AlertDialog>
        <AlertDialogTrigger asChild>
          <button className="bg-destructive text-white px-3 py-1 border-2 border-black font-bold text-sm
uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Delete</button>
        </AlertDialogTrigger>
        <AlertDialogContent className="bg-card border-4 border-white rounded-none">
          <AlertDialogHeader>
            <AlertDialogTitle className="font-display uppercase text-2xl text-destructive">Delete
Album?</AlertDialogTitle>
            <AlertDialogDescription className="text-white">This cannot be undone. Are you
sure?</AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel className="rounded-none border-2 border-black text-
black">Cancel</AlertDialogCancel>
            <AlertDialogAction onClick={() => handleDelete(album.id)} className="bg-destructive text-white
rounded-none border-2 border-black hover:bg-red-700">Delete</AlertDialogAction>

```



```

        </AlertDialogFooter>
        </AlertDialogContent>
    </AlertDialog>
</div>
</div>
))
):(
    <div className="text-center p-12 border-4 border-dashed border-gray-700 text-gray-500 font-display text-
2xl uppercase">No albums found.</div>
    )}
</div>
)}
</AdminLayout>
);
}

```

Manage Singles Page:

```
import { useState } from "react";
import { AdminLayout } from "@components/layout/AdminLayout";
import { useListSingles, useCreateSingle, useUpdateSingle, useDeleteSingle, getListSinglesQueryKey,
getGetSummaryQueryKey } from "@workspace/api-client-react";
import { useQueryClient } from "@tanstack/react-query";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { toast } from "sonner";
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription,
AlertDialogFooter, AlertDialogHeader, AlertDialogTitle, AlertDialogTrigger } from "@components/ui/alert-
dialog";
const singleSchema = z.object({
  title: z.string().min(1, "Required"),
  releaseDate: z.string().min(1, "Required"),
  description: z.string().min(1, "Required"),
  coverUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  listenUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
});
type SingleForm = z.infer<typeof singleSchema>;
export default function AdminSingles() {
  const queryClient = useQueryClient();
  const { data: singles = [], isLoading } = useListSingles();
  const createSingle = useCreateSingle();
  const updateSingle = useUpdateSingle();
  const deleteSingle = useDeleteSingle();
  const [editingId, setEditingId] = useState<number | null>(null);
  const [isFormOpen, setIsFormOpen] = useState(false);
  const form = useForm<SingleForm>({
    resolver: zodResolver(singleSchema),
    defaultValues: { title: "", releaseDate: "", description: "", coverUrl: "", listenUrl: "" }
  });
  const openCreate = () => {
```

```

setEditingId(null);
form.reset({ title: "", releaseDate: "", description: "", coverUrl: "", listenUrl: "" });
setIsFormOpen(true);
};

const openEdit = (single: any) => {
  setEditingId(single.id);
  const dateObj = new Date(single.releaseDate);
  const tzoffset = dateObj.getTimezoneOffset() * 60000;
  const localISOTime = (new Date(dateObj.getTime() - tzoffset)).toISOString().slice(0,16);
  form.reset({
    title: single.title,
    releaseDate: localISOTime,
    description: single.description,
    coverUrl: single.coverUrl || "",
    listenUrl: single.listenUrl || ""
  });
  setIsFormOpen(true);
};

const onSubmit = (data: SingleForm) => {
  const payload = {
    ...data,
    releaseDate: new Date(data.releaseDate).toISOString(),
    coverUrl: data.coverUrl || null,
    listenUrl: data.listenUrl || null,
  };
  if (editingId) {
    updateSingle.mutate({ id: editingId, data: payload }, {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: getListSinglesQueryKey() });
        queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
        toast.success("Single updated!");
        setIsFormOpen(false);
      },
      onError: (err) => toast.error(err.message)
    });
  } else {
    createSingle.mutate({ data: payload }, {

```

```

onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: getListSinglesQueryKey() });
  queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
  toast.success("Single created!");
  setIsFormOpen(false);
},
onError: (err) => toast.error(err.message)
});
}
};

const handleDelete = (id: number) => {
  deleteSingle.mutate({ id }, {
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: getListSinglesQueryKey() });
      queryClient.invalidateQueries({ queryKey: getGetSummaryQueryKey() });
      toast.success("Single deleted.");
    },
    onError: (err) => toast.error(err.message)
  });
};

return (
  <AdminLayout>
    <div className="flex justify-between items-center mb-8">
      <h2 className="font-display text-4xl text-white uppercase">Manage Singles</h2>
      {!isFormOpen && (
        <button onClick={openCreate} className="bg-white text-black border-2 border-black font-bold uppercase
px-4 py-2 shadow-[4px_4px_0_0_#fff] hover:translate-y-[2px] hover:shadow-[2px_2px_0_0_#fff]">
          + Add Single
        </button>
      )}
    </div>
    {isFormOpen ? (
      <div className="bg-card border-4 border-white p-6 shadow-[8px_8px_0_0_#fff] mb-8 rotate-[0.5deg]">
        <h3 className="font-display text-2xl uppercase mb-4">{editingId ? 'Edit Single' : 'New Single'}</h3>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

```

```

    <FormField control={form.control} name="title" render={({ field }) => (
      <FormItem><FormLabel>Single Title</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="releaseDate" render={({ field }) => (
      <FormItem><FormLabel>Release Date</FormLabel><FormControl><Input type="datetime-local"
{...field} className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="coverUrl" render={({ field }) => (
      <FormItem><FormLabel>Cover Art URL</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="listenUrl" render={({ field }) => (
      <FormItem><FormLabel>Streaming URL</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
  </div>
  <FormField control={form.control} name="description" render={({ field }) => (
    <FormItem><FormLabel>Description</FormLabel><FormControl><Textarea rows={4} {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
  )} />
  <div className="flex gap-4 pt-4">
    <button type="submit" disabled={createSingle.isPending || updateSingle.isPending} className="bg-
white text-black font-bold uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-
translate-y-1">
      Save Single
    </button>
    <button type="button" onClick={() => setIsFormOpen(false)} className="bg-gray-300 text-black font-
bold uppercase px-6 py-2 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-translate-y-1">
      Cancel
    </button>
  </div>
</form>
</Form>
</div>
):(
  <div className="space-y-4">

```

```

{isLoading ? (
  <div className="flex justify-center p-8"><div className="animate-spin h-8 w-8 border-4 border-white
border-t-transparent rounded-full"></div></div>
) : singles.length > 0 ? (
  singles.sort((a,b) => new Date(b.releaseDate).getTime() - new Date(a.releaseDate).getTime()).map(single =>
(
  <div key={single.id} className="flex flex-col sm:flex-row sm:items-center justify-between bg-black
border-2 border-white p-4 hover:border-white transition-colors">
    <div className="flex gap-4 items-center">
      {single.coverUrl && <img src={single.coverUrl} alt={single.title} className="w-16 h-16 object-cover
border-2 border-white" />}
    <div>
      <div className="text-white font-bold text-sm tracking-widest">{new
Date(single.releaseDate).toLocaleDateString()}</div>
      <div className="text-xl font-display uppercase">{single.title}</div>
    </div>
  </div>
  <div className="flex gap-2 mt-4 sm:mt-0">
    <button onClick={() => openEdit(single)} className="bg-white text-black px-3 py-1 border-2 border-
black font-bold text-sm uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Edit</button>
    <AlertDialog>
      <AlertDialogTrigger asChild>
        <button className="bg-destructive text-white px-3 py-1 border-2 border-black font-bold text-sm
uppercase shadow-[2px_2px_0_0_#000] hover:-translate-y-1">Delete</button>
      </AlertDialogTrigger>
      <AlertDialogContent className="bg-card border-4 border-white rounded-none">
        <AlertDialogHeader>
          <AlertDialogTitle className="font-display uppercase text-2xl text-destructive">Delete
Single?</AlertDialogTitle>
          <AlertDialogDescription className="text-white">This cannot be undone. Are you
sure?</AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel className="rounded-none border-2 border-black text-
black">Cancel</AlertDialogCancel>
          <AlertDialogAction onClick={() => handleDelete(single.id)} className="bg-destructive text-white
rounded-none border-2 border-black hover:bg-red-700">Delete</AlertDialogAction>

```

```

        </AlertDialogFooter>
        </AlertDialogContent>
    </AlertDialog>
</div>
</div>
))
):(
    <div className="text-center p-12 border-4 border-dashed border-gray-700 text-gray-500 font-display text-
2xl uppercase">No singles found.</div>
    )}
</div>
)}
</AdminLayout>
);
}

```

About/Info Page:

```
import { useEffect } from "react";
import { AdminLayout } from "@components/layout/AdminLayout";
import { useGetAbout, useUpdateAbout, getGetAboutQueryKey } from "@workspace/api-client-react";
import { useQueryClient } from "@tanstack/react-query";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { toast } from "sonner";
import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
const aboutSchema = z.object({
  bandName: z.string().min(1, "Required"),
  tagline: z.string().min(1, "Required"),
  story: z.string().min(1, "Required"),
  hometown: z.string().min(1, "Required"),
  foundedYear: z.coerce.number().int().min(1900),
  photoUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  youtubeUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  spotifyUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
  instagramUrl: z.string().url("Must be a valid URL").optional().or(z.literal("")),
});
type AboutForm = z.infer<typeof aboutSchema>;
export default function AdminAbout() {
  const queryClient = useQueryClient();
  const { data: about, isLoading } = useGetAbout();
  const updateAbout = useUpdateAbout();
  const form = useForm<AboutForm>({
    resolver: zodResolver(aboutSchema),
    defaultValues: {
      bandName: "", tagline: "", story: "", hometown: "", foundedYear: 2020,
      photoUrl: "", youtubeUrl: "", spotifyUrl: "", instagramUrl: ""
    }
  });
  useEffect(() => {
    if (about) {
```



```

form.reset({
  bandName: about.bandName,
  tagline: about.tagline,
  story: about.story,
  hometown: about.hometown,
  foundedYear: about.foundedYear,
  photoUrl: about.photoUrl || "",
  youtubeUrl: about.youtubeUrl || "",
  spotifyUrl: about.spotifyUrl || "",
  instagramUrl: about.instagramUrl || "",
});
}
}, [about, form]);
const onSubmit = (data: AboutForm) => {
  updateAbout.mutate({
    data: {
      ...data,
      photoUrl: data.photoUrl || null,
      youtubeUrl: data.youtubeUrl || null,
      spotifyUrl: data.spotifyUrl || null,
      instagramUrl: data.instagramUrl || null,
    }
  }, {
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: getGetAboutQueryKey() });
      toast.success("About info updated!");
    },
    onError: (err) => toast.error(err.message)
  });
};

return (
  <AdminLayout>
    <h2 className="font-display text-4xl text-accent uppercase mb-8">Edit About Info</h2>
    {isLoading ? (
      <div className="flex justify-center p-8"><div className="animate-spin h-8 w-8 border-4 border-accent
border-t-transparent rounded-full"></div></div>

```

```

): (
  <div className="bg-card border-4 border-white p-6 shadow-[8px_8px_0_0_#fef6a7] -rotate-[0.5deg]">
    <Form {...form}>
      <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-6">
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
          <FormField control={form.control} name="bandName" render={({ field }) => (
            <FormItem><FormLabel>Band Name</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
          )} />
          <FormField control={form.control} name="tagline" render={({ field }) => (
            <FormItem><FormLabel>Tagline</FormLabel><FormControl><Input {...field} className="bg-black
border-white" /></FormControl><FormMessage /></FormItem>
          )} />
          <FormField control={form.control} name="hometown" render={({ field }) => (
            <FormItem><FormLabel>Hometown</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
          )} />
          <FormField control={form.control} name="foundedYear" render={({ field }) => (
            <FormItem><FormLabel>Founded Year</FormLabel><FormControl><Input type="number" {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
          )} />
        </div>
        <FormField control={form.control} name="story" render={({ field }) => (
          <FormItem><FormLabel>Story / Bio</FormLabel><FormControl><Textarea rows={8} {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
        )} />
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 pt-4 border-t-2 border-dashed border-gray-600">
          <FormField control={form.control} name="photoUrl" render={({ field }) => (
            <FormItem><FormLabel>Photo URL</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
          )} />
          <FormField control={form.control} name="youtubeUrl" render={({ field }) => (
            <FormItem><FormLabel>YouTube URL</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
          )} />
          <FormField control={form.control} name="spotifyUrl" render={({ field }) => (

```

```

        <FormItem><FormLabel>Spotify URL</FormLabel><FormControl><Input {...field} className="bg-
black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
    <FormField control={form.control} name="instagramUrl" render={({ field }) => (
        <FormItem><FormLabel>Instagram URL</FormLabel><FormControl><Input {...field}
className="bg-black border-white" /></FormControl><FormMessage /></FormItem>
    )} />
</div>
    <button type="submit" disabled={updateAbout.isPending} className="bg-accent text-black font-bold
uppercase px-8 py-3 border-2 border-black shadow-[4px_4px_0_0_#000] hover:-translate-y-1 w-full mt-4 text-lg">
        Save About Info
    </button>
</form>
</Form>
</div>
    )}
</AdminLayout>
);
}

```

10.2 BIBLIOGRAPHY

Bierman, G., Abadi, M., & Torgersen, M. (2014). *TypeScript: An annotated JavaScript*. Microsoft Research. <https://www.typescriptlang.org/>

Collet, T. (2023). *Drizzle ORM: TypeScript ORM for SQL databases* (Version 0.30). Drizzle Team. <https://orm.drizzle.team/>

Evans, Z. (2023). *esbuild: An extremely fast JavaScript bundler* (Version 0.21). <https://esbuild.github.io/>

Express.js Contributors. (2024). *Express: Fast, unopinionated, minimalist web framework for Node.js* (Version 5). OpenJS Foundation. <https://expressjs.com/>

Node.js Contributors. (2024). *Node.js: Cross-platform JavaScript runtime environment* (Version 24). OpenJS Foundation. <https://nodejs.org/>

pnpm Contributors. (2024). *pnpm: Fast, disk space efficient package manager* (Version 10). <https://pnpm.io/>

PostgreSQL Global Development Group. (2024). *PostgreSQL: The world's most advanced open source relational database* (Version 16). <https://www.postgresql.org/>

React Contributors. (2024). *React: The library for web and native user interfaces* (Version 19). Meta Open Source. <https://react.dev/>

Replit Inc. (2024). *Replit: Build software collaboratively with the power of AI*. <https://replit.com/>

shadcn. (2024). *shadcn/ui: Beautifully designed components built with Radix UI and Tailwind CSS*. <https://ui.shadcn.com/>

Tailwind Labs. (2024). *Tailwind CSS: A utility-first CSS framework* (Version 4). <https://tailwindcss.com/>

TanStack Contributors. (2024). *TanStack Query: Powerful asynchronous state management for TypeScript* (Version 5). <https://tanstack.com/query/>

Vite Contributors. (2024). *Vite: Next generation frontend tooling* (Version 7). <https://vitejs.dev/>

Wouter Contributors. (2024). *Wouter: A minimalist-friendly routing library for React* (Version 3). <https://github.com/molefrog/wouter>

Zod Contributors. (2024). *Zod: TypeScript-first schema validation with static type inference* (Version 3). <https://zod.dev/>